

Code Assessment of the MetaDEX Application Layer Smart Contracts

August 29, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	6
4	Trust Model	13
5	System Considerations	21
6	Terminology	25
7	Findings	26
8	Open Questions	50
9	Limitations and use of report	51



1 Executive Summary

Dear Velodrome Team,

Thank you for trusting us to help Velodrome with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of MetaDEX Application Layer according to [Scope](#) to support you in forming an opinion on their security risks.

Velodrome implements the application layer of MetaDEX, the emission and reward infrastructure that runs on each leaf chain together with the AMM pools that feed it. Gauges distribute the token emissions to staked LP tokens, and voting rewards contracts pass the pool fees and third-party incentives on to veNFT holders by vote weight. The pools come in a constant-product and a stable-swap variant, price their swaps through pluggable fee modules, and record their activity in on-chain tapes. A metarouter is the unified access point for user interactions with both, including those arriving from another chain.

The most critical subjects covered in our audit are reward and emission accounting, permissionless execution and robustness of oracles. Security regarding reward and emission accounting is good. The old design pushed vote updates to the voting rewards manager through a `try / catch` that an attacker could force to revert, which caused [Attacker-Caused Missing Gauge Rate Windows](#) and [A Failed Checkpoint Keeps a veNFT's Weight on Two Gauges](#). It has been replaced by a pull-based design that reverts on failure. Security regarding permissionless execution is improvable, since a relayer can forfeit emission, see [Malicious Relayer Can Forfeit All Accrued Emissions](#). Security regarding robustness of oracles is improvable. Data generated by the AMMs is expected to be used in future upgrades to improve the efficiency of the staking mechanism, so manipulation is a concern even with no consumers today, see [Anyone Can Distort the Staked Liquidity Value in the Pool Tape](#).

The general subjects covered are error handling, integration testing, and documentation & specification. Security regarding error handling is improvable. A number of external calls catch reverts. While this can improve liveness it also offers ample attack surface for attackers to trigger the failure branch by, for example, choosing a low gas limit, see [Insufficient Gas Griefing to Trigger the Else Branch](#). The testing suite is insufficient, as it mostly consists of unit tests and no extensive integration tests exist. Security regarding documentation & specification is good, with extensive design documents available, though not all behavior is documented, see [The Fee Resolution Order Is Undocumented](#).

In contrast to a regular audit process, the codebase was not fixed at a single commit but evolved continuously during the review and as such the latest version of the code was not covered extensively. Together with the complexity of the design and the improvable testing suite, we believe an above-average number of undiscovered issues remain.

In summary, we find that the codebase provides an improvable level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the MetaDEX Application Layer repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	31 July 2026	90c05dcaea50bc00e7f19b56cb83600fa66083e7	Initial review
2	20 August 2026	23762e23a7eccefe4f3d807c0cdf31fec5dc0818	Extended Scope

Version 1 and **Version 2** were covered by a single continuous review with no fix round in between. A finding tagged **Version 2** may therefore have been present in **Version 1** as well. The tag names the version the finding is written against and is chosen for readability. It does not indicate when or how the issue was found. Further, some patches for findings are already partially applied, but are not in the current report due to time constraints.

For the Solidity smart contracts, the compiler version `0.8.30` was chosen.

The scope covers all contracts of the `factories`, `fees`, `gauge-creation`, `gauges`, `hooks`, `metarouter`, `pools` and `rewards` folders, together with selected files of the `libraries` folder:

```
V3/src/
├── factories/
│   ├── FactoryRegistry.sol
│   ├── GaugeFactory.sol
│   ├── PoolFactory.sol
│   ├── StablePoolFactory.sol
│   └── VolatilePoolFactory.sol
├── fees/
│   ├── CustomFeeModule.sol
│   ├── DiscountRegistry.sol
│   └── PriorityFeeMevTaxModule.sol
├── gauge-creation/
│   ├── DefaultGaugeCreationModule.sol
│   ├── GaugeManager.sol
│   └── GovernanceCreationModule.sol
├── gauges/
│   ├── Gauge.sol
│   └── V2Gauge.sol
├── hooks/
│   ├── dynamic/
│   │   ├── DynamicSwapFeeHook.sol
│   │   └── libraries/
│   │       └── TransientMevTaxLib.sol
├── libraries/
│   ├── ParamsLib.sol
│   ├── PoolOracle.sol
│   ├── ProtocolConstants.sol
│   ├── SafeCastLibrary.sol
│   └── VolatilityRingLibrary.sol
```

```

├── metarouter/
│   ├── Metarouter.sol
│   ├── TransientTracking.sol
│   └── libraries/
│       ├── ClaimsLib.sol
│       ├── ClPositionLib.sol
│       ├── Commands.sol
│       ├── CrosschainLib.sol
│       ├── FundsLib.sol
│       ├── LiquidityLib.sol
│       ├── MetarouterState.sol
│       ├── PaymentsLib.sol
│       └── StakingLib.sol
├── pools/
│   ├── Pool.sol
│   ├── PoolFactoryIndexation.sol
│   ├── PoolFees.sol
│   ├── StablePool.sol
│   ├── VolatilePool.sol
│   └── tape/
│       ├── BasePoolTape.sol
│       ├── ClPoolTape.sol
│       └── PoolTape.sol
└── rewards/
    ├── FeeDistribution.sol
    ├── IncentiveStreaming.sol
    ├── VotingCheckpoints.sol
    ├── VotingRewardsFactory.sol
    └── VotingRewardsManager.sol

```

The integration with the relay in `StakeRelayLib` is explicitly out of scope of this review:

```

V3/src/
├── metarouter/
│   └── libraries/
│       └── StakeRelayLib.sol

```

Therefore, the metarouter commands `CREATE_STAKE` and `DEPOSIT_RELAY` are also excluded from the review.

2.1.1 Excluded from scope

All contracts not explicitly listed above are out of scope of this review.

Third-party libraries, namely OpenZeppelin, Solady and Nomad, are out of scope and assumed to work correctly. Further, while the integration with Hyperlane is in scope of this review, the Hyperlane contracts themselves are not. Their security depends on a correct configuration, which is also not in scope of this review.

Further, the economic design and the correct choice of parameters are out of scope of this review.

The contracts are expected to be deployed on the Optimism and Base L2s. Other chains are not covered by this review, and the differences in their behavior must be carefully considered before deploying there.

3 System Overview

This system overview describes the V3 contracts as defined in the [Assessment Overview](#).

3.1 Architecture

This document covers the V3 emission and reward infrastructure deployed on leaf chains, together with the AMM pool and dynamic swap fee layer. The in-scope contracts implement:

1. LP token staking and pull-based emission distribution
2. vote-weight-proportional fee and incentive distribution to veNFT holders
3. constant-product and stable-swap AMM pools with trading fee accounting
4. volatility-responsive dynamic swap fees with MEV taxation for CL pools
5. composable batch-execution routing
6. gauge lifecycle management and factory registry infrastructure
7. the per-chain emission coordinator (`LeafVoter`)

Cross-chain bridging is not covered. The description of `LeafVoter` is limited to its role as the gauge and reward coordination layer.

3.2 AMM Pools

3.2.1 Pool (Abstract Base)

`Pool` is the abstract base shared by `StablePool` and `VolatilePool`. It mints ERC20 LP tokens pro-rata to deposited liquidity and burns them on withdrawal. Swaps use an optimistic transfer pattern: output tokens are released first, an optional `IPoolCallee.hook()` callback is invoked (enabling flash loans), and the required input amount is verified afterward by checking that the invariant `_k` has not decreased. A fee, queried per pool from `IPoolFactory.getFee()`, is deducted from input amounts and transferred to a companion `PoolFees` contract. Reserves are tracked in storage and updated via `_update()`, which also maintains cumulative reserve time-accumulators (stored as `uint256`) and writes to a circular TWAP observation buffer at most once per `PERIOD_SIZE` (1 minute). `getAmountOutWithTotalFee(amountIn, tokenIn)` and `getAmountInWithTotalFee(amountOut, tokenOut)` are view quotation helpers that include the pool's total contextual fee.

3.2.2 StablePool

`StablePool` implements the stable-swap invariant $x^3y + xy^3 = k$, normalized to token decimals. Output amounts are solved via Newton-Raphson iteration. Initial liquidity requires equal (decimal-normalized) token amounts and a minimum `k` of `10^10`. A post-burn check prevents the invariant from reaching zero.

3.2.3 VolatilePool

`VolatilePool` implements the constant-product invariant $xy = k$.

3.2.4 PoolFees

`PoolFees` holds fee tokens collected from the pool and releases them via `claimFeesFor()`.

3.2.5 PoolFactory (Abstract Base)

`PoolFactory` is the abstract base for both `StablePoolFactory` and `VolatilePoolFactory`. It inherits `PoolFactoryIndexation`, which maintains three enumerable indexes updated on every pool deployment: all deployed pools (`poolsIndex`), pools grouped by constituent token (`poolByTokenIndex`), and the full token set (accessible via `tokenIndexPaginated` and `tokenIndexLength`). All three expose paginated views. The factory deploys pool clones deterministically using a salt of `keccak256(token0, token1)`. It maintains a global default fee (max 3%), overridable per pool via an optional `IFeeModule`. A `pauser` address may halt all pool creation. When a `poolTape` address is configured, per-swap metrics are forwarded via `recordPoolTape()` using `excessivelySafeCall` with a configurable gas limit (`poolTapeGasLimit`), ensuring tape failures do not revert swaps.

3.2.6 PoolTape

`PoolTape` records cumulative per-swap metrics for each V2 pool. On every swap, the pool calls `PoolFactory.recordPoolTape()`, which forwards a `PoolTapeData` struct to `PoolTape.record()`. The contract maintains per-pool `Accumulator` state tracking cumulative fee, volume, MEV volume, MEV fee (both token0 and token1), and swap count. At configurable cadence intervals, accumulator snapshots are committed to a circular observation buffer of up to 65,535 slots per pool. Slots are allocated one at a time on growth to avoid uninitialized reads. Historical queries via `observe(pool, secondsAgo[])` interpolate linearly between the two surrounding committed observations.

3.3 Dynamic Fees

3.3.1 DynamicSwapFeeHook

`DynamicSwapFeeHook` is an `ISwapHook` implementation for CL pools that provides a volatility-responsive dynamic fee. On `beforeSwap`, it resolves a fee and stores it in transient storage alongside a `toxic` flag. On `afterSwap`, it uses those cached values to split fee attribution into base and MEV components and records the swap to `CLPoolTape`.

Fee resolution proceeds in layers:

1. **Base fee:** the per-pool `baseFee`. Falls back to the factory tick-spacing fee when unset.
2. **Dynamic volatility component:** on the first swap in a block, a volatility signal is read from `CLPoolTape` and scaled by a per-pool `scalingFactor` (defaulting to `defaultScalingFactor`). The sum `baseFee + dynamicFee` is capped at the per-pool `feeCap` (defaulting to `defaultFeeCap`). The result is stored in `blockFee[pool][block.number]` and reused for all subsequent swaps in that block.
3. **Discount:** `DiscountRegistry.getDiscount(pool, caller)` reduces the block-wide fee for whitelisted non-EOA callers (`tx.origin != caller`).
4. **Initial fee:** when `initialFeeEnabled` is set for a pool, the first transaction in the block may use a separately configured `initialFee` for all swaps within that transaction, overriding the block-wide dynamic fee.

The `mevTaxModule` surcharge is added on top in `beforeSwap` and separated back out in `afterSwap` by subtracting `dynamicFee` from the final `fee`, giving `mevFee = totalFee - dynamicFee`. If `mevFee > 0`, the fees are split into base and MEV buckets for recording.

3.3.2 PriorityFeeMevTaxModule

`PriorityFeeMevTaxModule` implements `IMevTaxModule`. `mevTax()` returns `(mevFee, isToxic)`. `isToxic` is `true` when `tx.gasprice > block.basefee`. `mevFee` is `min(priorityFeeMultiplier * max(0,`

`tx.gasprice - max(baseFeeFactor * block.basefee, minThreshold)), feeCeiling)`, truncated to `uint24`. A priority fee at or below the threshold returns zero.

3.3.3 DiscountRegistry

`DiscountRegistry` maps accounts to fee discounts, each consisting of a chain-wide value and a per-pool override set. `getDiscount(pool, caller)` returns the larger of the two applicable values, but only when `tx.origin != caller`. Direct EOA transactions receive no discount. Used by `DynamicSwapFeeHook` to reduce the block-wide base fee for whitelisted protocol contracts.

3.4 V3 Gauges

3.4.1 Gauge (Abstract Base)

`Gauge` is the abstract base for V3 LP staking gauges. It manages LP token deposits and withdrawals, claimable emission balances per staker, approved-for-claim delegates, and operator approvals for claim delegation. It defines the penalty and referral helper functions `_applyPenalty()` and `_applyReferral()`.

3.4.2 V2Gauge

`V2Gauge` is the concrete V3 gauge for V2 AMM LP tokens. It tracks `lastCumulativeRewardShare` (a cursor into `LeafVoter.GaugeState.ceiling`) and `rewardPerTokenStored`. On any staker interaction, `_advanceRewards()` calls `LeafVoter.settleGauge()`, computes the delta `ceiling - lastCumulativeRewardShare`, and if stakers exist distributes it as `delta * PRECISION / totalSupply` added to `rewardPerTokenStored`. If no stakers held the gauge over the settled span, the entire delta is forfeited via `forfeitEmissions`. `earned()` uses `LeafVoter.projectedCumulativeRewardShare()` for a gas-free preview. On emission claim, `_claimEmissions()` calls `LeafVoter.forfeitEmissions()` to return unused or penalized emissions as surplus, then calls `LeafVoter.mintEmissions(recipients, amounts)` to mint tokens for the staker and any referral. Early unstake is subject to a configurable penalty loaded from `GaugeFactory.effectivePenaltyConfig()`: if the staker's deposit has been held for fewer than `minStakeBlocks` blocks, a `penaltyRate` fraction of unclaimed emissions is forfeited rather than paid out. An optional referral address may be configured per gauge to receive a PIPS-denominated share of each emission claim.

3.4.3 GaugeFactory

`GaugeFactory` deploys `V2Gauge` clones and manages gauge-level emission configuration. Clone salts are derived from `keccak256(pool)`. The factory exposes four administrative roles:

1. `CAP_ADMIN_ROLE` for setting default and per-gauge emission caps
2. `REFERRAL_ADMIN_ROLE` for configuring referral addresses and share fractions per gauge
3. `PENALTY_ADMIN_ROLE` for setting chain-wide minimum stake blocks and penalty rates
4. `CAP_OPERATOR_ROLE` for adjusting per-gauge caps within the range `[operatorMinCap, operatorMaxCap]`

The `EMERGENCY_COUNCIL_ROLE` (resolved through `LeafVoter`) may additionally force any gauge's cap to zero. Per-gauge emission caps are stored as per-second limits. A sentinel value (`DEFAULT_CAP_INDICATOR`) causes `emissionCap(gauge)` to return the factory's `defaultCap`. `setEmissionCap` always calls `settleGauge` before applying any cap change. Setting a cap to zero suspends emissions: `GaugeFactory` calls `flushFees()` on the gauge's `VotingRewardsManager` and `settleGauge()` on `LeafVoter`, closing out accruals before the gauge goes dark. Per-gauge `minStakeBlocks` overrides are configurable via `setMinStakeBlocks`. `effectivePenaltyConfig(gauge)` resolves the per-gauge effective config. A global `maxShareCap` (default 50,000 PIPS) bounds any referral share.

3.5 V3 Rewards

3.5.1 VotingCheckpoints (Abstract Base)

`VotingCheckpoints` maintains veNFT voting weight allocations as a slope/bias point history, adapted from the `VotingEscrow` model. Each veNFT's allocation is stored as a `(slope, bias)` pair that decays linearly to zero at stake expiry. Permanent (non-decaying) stakes are tracked separately in `permanentStakeBalance`. Slope changes scheduled at future stake expiries are tracked in `slopeChanges[timestamp]`. The global supply and per-NFT balance at any past timestamp are recoverable via binary search over checkpointed point histories. `_checkpoint(tokenId, allocated, stakeEnd, data)` records a new allocation, while `_reset(tokenId)` removes an existing one, canceling any scheduled future slope change if the stake has not yet expired.

3.5.2 FeeDistribution (Abstract Base)

`FeeDistribution` extends `VotingCheckpoints` to distribute pool trading fees proportionally to voting power. It maintains a running `feeRewardPerVotingPower` accumulator (a `FeeSnapshot` struct holding `feeReward0`, `feeReward1`, and their time-weighted counterparts `feeReward0xTime`, `feeReward1xTime`). `_notifyFeesAmount(pending0, pending1)` credits the supplied fee amounts plus any buffered residual to the accumulator. Fees below the precision threshold relative to total supply are buffered rather than credited immediately, preventing fractional-token dust from accumulating rounding error. A zero-cap gauge's buffer is not flushed. The time-weighted components enable pro-rata computation for decaying stakes whose effective weight varies continuously over the reward period. Snapshots of the fee accumulator are recorded at each global reward checkpoint index via `_snapshotFeeAccumulator()`.

3.5.3 IncentiveStreaming (Abstract Base)

`IncentiveStreaming` manages permissionless incentive programs: any address may call `createIncentiveProgram(token, amount, start, duration)` to fund a fixed-rate token stream of at least 7 days directed at voters for a pool. The deposited tokens are held in the contract and streamed at a constant rate (amount/duration) over the program window. Token eligibility is gated by a `ITokenRegistry` whitelist. Per-program and per-veNFT claim state is tracked to support lazy multi-program claiming. Snapshots of supply accumulator values are recorded at global checkpoints for use in pro-rata incentive computation. A `sweep(programId)` function allows the program creator to reclaim tokens that accrued during periods of zero voting supply.

3.5.4 VotingRewardsManager

`VotingRewardsManager` is the concrete reward contract. It inherits `FeeDistribution`, `IncentiveStreaming`, and the shared `VotingCheckpoints` base, implementing `_createGlobalRewardPoints()` to maintain the unified global checkpoint history used by all three layers. The contract is deployed with references to a gauge, the associated `GaugeFactory` (for cap queries), and the pool's two fee tokens.

`checkpoint(tokenId, allocated, stakeEnd, data)` is called by `LeafVoter` when a veNFT's vote allocation changes. `flushFees()` is callable by `GaugeFactory` to force a fee accumulator update outside of a normal claim flow (used when a gauge's cap is set to zero). `claimFees(tokenId, recipient, maxCheckpoints)` and `claimIncentives(tokenId, recipient, programId, maxCheckpoints)` allow veNFT holders or their approved delegates to lazily claim accrued fees and incentives. The `_maxCheckpoints` parameter bounds the number of checkpoints processed per call, preventing unbounded iteration and enabling multi-call claiming for tokens with long histories. `claimIncentives` operates on one `programId` per call. `earnedFees(tokenId, maxCheckpoints)` and `earnedIncentives(tokenId, programId, maxCheckpoints)` are view functions estimating claimable amounts over the same bounded range. `feeClaimState` and `incentiveClaimState` expose the per-token

claim cursor. Native token rewards are automatically unwrapped via the configured `wrappedNative` address on payout. `advanceGlobalPoints()` is permissionless and drives the global checkpoint forward without any user-specific claim.

3.5.5 VotingRewardsFactory

`VotingRewardsFactory` deploys one `VotingRewardsManager` per gauge, restricted to the `voter` address. It passes the gauge address, gauge factory address, and initial reward token set (the pool's `token0` and `token1`) into each deployment.

3.6 Factory and Registry

3.6.1 FactoryRegistry

`FactoryRegistry` links gauge factories to target (pool) factories, records `targetToFactory` and `gaugeToFactory` relationships, and maintains an enumerable set of approved Metarouter addresses. Pool factory registration can be seeded by a `targetFactoryAdmin` before the `LeafVoter` is wired, supporting a lite-launch scenario where pools are deployed before the voting system is live. `registerGauge` writes each gauge's record exactly once, requiring its target to originate from the linked target factory. `emissionCap(gauge)` delegates to the gauge's factory. `gaugeToRewards` / `rewardsToGauge` are used by gauges and `LeafVoter` for rewards lookups.

3.6.2 GaugeManager and Creation Modules

`GaugeManager` mediates the gauge lifecycle. It validates factory and target provenance against `FactoryRegistry`, deploys through the appropriate gauge factory, and calls `LeafVoter.registerGauge` and `LeafVoter.activateGauge`. Gauges may be registered in an inactive state and activated later via a separate `activateGauge` call. Two pluggable creation modules are available: `DefaultGaugeCreationModule` (permissionless, gated by a token registry whitelist) and `GovernanceCreationModule` (governance-gated).

3.7 Metarouter

`Metarouter` is a permissionless batch-execution router. An outer `execute(commands, inputs, deadline)` call opens a custody frame in transient storage and dispatches a sequence of command bytes. At closure, all tracked ERC20 balances and surplus native ETH are swept back to the original caller. Any tracked NFT not released during the batch causes closure to revert. Each command byte encodes a command type and a `FLAG_ALLOW_REVERT` flag. A flagged command runs in a child call frame whose failure is silently swallowed without aborting the batch.

Supported command groups:

- **Swaps:** exact-input and exact-output routes over CL and V2 pool chains, validated against `FactoryRegistry`. CL routes use the `uniswapV3SwapCallback` pattern with transient-storage authorization of the callback caller. V2 exact-out quotes each hop backward from the pool's own `getAmountInWithTotalFee`.
- **Payments:** `FUND_ERC20`, `SWEEP`, `TRANSFER`, `TRANSFER_NFT`, `WRAP_ETH`, `UNWRAP_WETH` for routing assets in and out of the custody frame.
- **CL positions:** `MINT_CL_POSITION`, `INCREASE_CL_LIQUIDITY`, `DECREASE_CL_LIQUIDITY`, `COLLECT_CL_FEES`, `BURN_CL_POSITION` over the canonical `NonfungiblePositionManager`.

- **V2 liquidity:** `ADD_LIQUIDITY` and `REMOVE_LIQUIDITY` through pool factories verified by `FactoryRegistry`.
- **Gauge staking:** `STAKE_GAUGE` and `UNSTAKE_GAUGE` for both V2 and CL gauges via `LeafVoter`. Approved Metarouters bypass the V2Gauge penalty-active guard on `depositFor`. `UNSTAKE_GAUGE` claims and sweeps the emission token to the batch.
- **CL LP-incentive staking:** `STAKE_LP_INCENTIVE`, `UNSTAKE_LP_INCENTIVE`, `WITHDRAW_LP_INCENTIVE` over the `ILpIncentive` staker contract.
- **Claims:** `CLAIM_GAUGE_REWARDS` forwards to the emission gauge. `CLAIM_LP_INCENTIVE_REWARDS` and `CLAIM_V2_POOL_FEES` do the same for LP incentives and pool fees.
- **Cross-chain:** `BRIDGE_TOKEN` and `EXECUTE_CROSS_CHAIN` dispatch through the `IInterchainAccountRouter`. These commands are disabled when the router is deployed without one.
- **Control flow:** `EXECUTE_SUB_PLAN` runs a nested batch whose revert is re-thrown to the caller. `BALANCE_CHECK` asserts a minimum balance mid-batch.

NFT custody is tracked by `(collection, tokenId)` pair in transient storage. `onERC721Received` accepts a position only while a batch is active, only from the canonical position manager, and only when the collection and token ID match the slot armed by the executing command. When `LEAF_VOTER` is not set (a lite deployment with no voting system), gauge staking and claim commands are disabled. When `ICA_ROUTER` is not set, cross-chain commands are disabled.

The `Relay` contract is out of scope. The Metarouter commands `CREATE_STAKE` and `DEPOSIT_RELAY`, as well as the `StakeRelayLib` library, are therefore also excluded.

3.8 LeafVoter (Emission Coordination)

`LeafVoter` is the per-chain emission coordinator that translates chain-level emission budgets and veNFT vote allocations into per-gauge emission entitlements. The following describes only its gauge and reward coordination behavior. Cross-chain message handling and bridging are excluded.

3.8.1 Per-Gauge Settlement

`settleGauge(gauge)` is the external entry point for gauges. It walks the gauge's voting-power point through elapsed time, integrates allocations against the chain-level emission rate, and credits the result to `GaugeState.ceiling`, a monotonically increasing cumulative reward share in token units. The gauge reads the delta since its own `lastCumulativeRewardShare` cursor to determine how much has accrued. `projectedCumulativeRewardShare(gauge)` replicates this walk read-only, projecting the unsettled tail at the current emission rate, so `V2Gauge.earned()` can return accurate values without writing state.

3.8.2 Emission Minting and Forfeiture

`mintEmissions(recipients, amounts)` is called by a registered gauge after computing the amounts owed to stakers. `LeafVoter` verifies the total does not exceed the gauge's `ceiling - claimed - surplus` headroom, increments `claimed`, mints the receipt token to `EMISSIONS_HANDLER`, and calls `IEmissionsHandler.handleEmissions()` for token delivery. `forfeitEmissions(amount)` is called by a gauge to return unused or penalized emissions as surplus. The reported amount is clamped to the claimable headroom before being added to both the gauge's `surplus` and the chain-level `surplusAccrued`. When the chain status is `Suspended` or `Sunset`, the emissions scalar is parked at zero so gauges accrue nothing over those periods. `mintEmissions` and `forfeitEmissions` remain open during `Sunset` so earned emissions stay claimable.

3.8.3 Reward Claiming

`claimRewards(tokenId, recipient, feeClaims, incentiveClaims)` is a multi-target convenience wrapper callable by the token's operator or the `LeafMessageOrchestrator`. It iterates `FeeClaim` and `IncentiveClaim` request lists, forwarding each to the corresponding `VotingRewardsManager`. Individual failures emit `FeeClaimFailed` / `IncentiveClaimFailed` and do not revert the batch.

3.9 Key System Interactions

3.9.1 Gauge Settlement and Emission Accrual

Root dispatches chain-level emission budgets to `LeafVoter`, which integrates the chain's emission rate over time into per-gauge entitlements stored as `GaugeState.ceiling`. When a `V2Gauge` staker interacts, `_advanceRewards()` calls `LeafVoter.settleGauge()` to advance the ceiling to the current block. The gauge diffs the returned value against its own `lastCumulativeRewardShare` cursor and distributes the accrued amount proportionally to staked LP tokens.

3.9.2 Emission Claim Flow

After crediting a staker's balance, `V2Gauge._claimEmissions()` calls `LeafVoter.forfeitEmissions(unusedAmount + penaltyAmount)` to return surplus, then calls `LeafVoter.mintEmissions(recipients, amounts)` with the staker address (and optionally the referral address) as recipients. `LeafVoter` verifies the headroom, mints the receipt token to `EMISSIONS_HANDLER`, and triggers delivery.

3.9.3 Fee Distribution to Voters

Trading fees accumulate in `PoolFees`. When the gauge's cap is set to zero or during a normal claim-preparation flow, `GaugeFactory.flushFees(gauge)` triggers `VotingRewardsManager.flushFees()`, which calls `gauge.pendingFees()` and passes the pending amounts to `_notifyFeesAmount()`. The fee accumulator is incremented by the per-supply increment, and the time-weighted counterpart is updated. veNFT holders later call `claimFees()` to receive their pro-rata share based on the vote weight they held during the relevant periods.

3.9.4 Incentive Programs and Claiming

Anyone may call `VotingRewardsManager.createIncentiveProgram()` to fund a stream with a token listed in the `TokenRegistry`. Voters later call `claimIncentives(tokenId, recipient, programId, maxCheckpoints)` to collect their share for an active program across a bounded number of past checkpoints. For periods with zero total voting supply, the program creator may recover the unclaimed tokens via `sweep(programId)`. Each checkpoint interval is priced from the voting power held at its final second rather than from the average across it, which credits permanent stakes a larger share of an interval than decaying stakes of the same starting weight.

4 Trust Model

4.1 Roles

Governance (`GOVERNANCE_ROLE` on `LeafVoter`)

- Trust level: Fully trusted.
- Permissions: Create gauges for any target without token-listing requirements via `GovernanceCreationModule`. Activate any registered gauge via `GaugeManager.activateGauge`, bypassing module ownership checks.
- Worst-case impact: Activate gauges for unvetted or malicious pools, directing protocol emissions to illegitimate targets. Because emissions are minted by `LeafVoter` on gauge request, no in-scope funds are at direct risk. The damage is economic.

Module Admin (`MODULE_ADMIN_ROLE` on `LeafVoter`)

- Trust level: Fully trusted.
- Permissions: Register and deregister creation modules in `GaugeManager`. Add and remove base assets and allowed gauge factories in `DefaultGaugeCreationModule`.
- Worst-case impact: Register a malicious creation module, which could forward arbitrary targets to `GaugeManager` and have gauges deployed for non-pool contracts. Removing all allowed gauge factories disables permissionless gauge creation.

Factory Registry Admin (`FACTORY_REGISTRY_ADMIN_ROLE` on `LeafVoter`)

- Trust level: Fully trusted.
- Permissions: Register and unregister factory pairs in `FactoryRegistry`. Register and unregister `Metarouter` addresses. Set the `TokenRegistry` pointer.
- Worst-case impact: Register a malicious factory pair, allowing gauge creation to record arbitrary contracts as gauges and targets. Registering a malicious `Metarouter` bypasses penalty checks on `V2Gauge.depositFor`. Replacing the `TokenRegistry` pointer can gate or ungate gauge activation and incentive creation for all tokens.

Emergency Council (`EMERGENCY_COUNCIL_ROLE` on `LeafVoter`)

- Trust level: Partially trusted.
- Permissions: Set any gauge's emission cap to zero in `GaugeFactory`, which also flushes the gauge's pending fees via `VotingRewardsManager.flushFees`.
- Worst-case impact: Permanently suppress emissions for any gauge. Fees already accrued are flushed and remain claimable by voters. No funds are lost.

GaugeFactory Cap Admin (`CAP_ADMIN_ROLE` on `GaugeFactory`)

- Trust level: Partially trusted.
- Permissions: Set the factory-wide default emission cap and the operator cap range. Set any individual gauge's emission cap to any value, including zero (which also flushes fees). Grant and revoke `CAP_OPERATOR_ROLE`.
- Worst-case impact: Set all gauge caps to zero, halting emissions across the entire factory and triggering fee flushes. Can also set caps to `type(uint128).max`, though emissions are bounded by the voter's allocation. `CAP_ADMIN_ROLE` is self-administering. `GuardedAccessControlEnumerable` prevents permanently losing the role by blocking removal of the last holder.

GaugeFactory Cap Operator (`CAP_OPERATOR_ROLE` on `GaugeFactory`)

- Trust level: Partially trusted.
- Permissions: Adjust an individual gauge's emission cap within the `[operatorMinCap, operatorMaxCap]` range or reset it to the default indicator. Cannot set caps to zero or outside the configured range.
- Assumptions: `operatorMinCap` and `operatorMaxCap` are set to sensible bounds by `CAP_ADMIN_ROLE`.
- Worst-case impact: Set a gauge cap to `operatorMaxCap` without governance intent, shifting emissions disproportionately. Economic impact only. No in-scope funds are stolen.

GaugeFactory Referral Admin (`REFERRAL_ADMIN_ROLE` on `GaugeFactory`)

- Trust level: Partially trusted.
- Permissions: Set the factory-wide maximum referral share cap (up to 100%). Set per-gauge referral recipient and share, overriding any creator-specified referral at creation time.
- Worst-case impact: Configure any gauge to redirect up to 100% of its emissions to an arbitrary address controlled by the referral admin, effectively stealing all emission yield from LP stakers.
`REFERRAL_ADMIN_ROLE` is self-administering and guarded against losing the last holder.

GaugeFactory Penalty Admin (`PENALTY_ADMIN_ROLE` on `GaugeFactory`)

- Trust level: Partially trusted.
- Permissions: Set the factory-wide penalty rate (up to 100%) and minimum stake block count (up to `MAX_MIN_STAKE_BLOCKS`). Set per-gauge minimum stake blocks independently.
- Worst-case impact: Configure a 100% penalty rate and a long minimum stake window, causing early-withdrawing stakers to forfeit all accrued emissions. Forfeited emissions return to the voter pool via `LeafVoter.forfeitEmissions`. `PENALTY_ADMIN_ROLE` is self-administering and guarded against losing the last holder.

DiscountRegistry Owner

- Trust level: Partially trusted.
- Permissions: Register per-account chain-wide and per-pool fee discounts up to 80% (`MAX_DISCOUNT`). Grant pool fee-floor exemptions. Two-step ownership transfer via `Ownable2Step`.
- Worst-case impact: Grant an 80% fee discount to a specific address, reducing swap fee revenue distributed to LPs. Grant fee-floor exemptions, allowing selected accounts to bypass minimum fee thresholds. Does not affect protocol emissions.

TokenRegistry Admin

- Trust level: Partially trusted.
- Permissions: List and delist tokens in `TokenRegistry`. Listing a token enables gauge activation via `DefaultGaugeCreationModule.activate` and incentive program creation via `IncentiveStreaming.createIncentiveProgram`. Delisting has no retroactive effect on already-created programs.
- Worst-case impact: List a malicious or low-liquidity token, enabling incentive programs funded by that token. Program creators can only recover their own deposited funds via `sweep`. No other user's claims are affected. Delist a legitimate token to block new gauge activations and incentive programs for that token without affecting existing ones.

PoolFactory Pool Admin (`poolAdmin`)

- Trust level: Partially trusted.
- Permissions: Rename existing pools via `setName` and `setSymbol`.

- Worst-case impact: Rename pools to misleading labels, potentially causing off-chain UI confusion. No on-chain financial impact. `poolAdmin` is a simple address slot transferable to zero (irreversible).

PoolFactory Pauser (`pauser`)

- Trust level: Partially trusted.
- Permissions: Pause and unpause all swap operations across all pools created by the factory.
- Worst-case impact: Indefinitely pause swaps, blocking LP withdrawals (which still function) and all trading activity. Pool LPs retain their position and can still burn LP tokens to withdraw underlying tokens during a pause.

PoolFactory Fee Manager (`feeManager`)

- Trust level: Partially trusted.
- Permissions: Set the factory-wide default swap fee (up to 3%). Set or replace the `IFeeModule` contract (validated by checking `factory()` on the module). Set or replace the `IMevTaxModule` contract. Transfer the `feeManager` role.
- Assumptions: The `IFeeModule` and `IMevTaxModule` contracts set are non-malicious. Pool calls to both are wrapped in `excessivelySafeCall`, capping gas and limiting return data.
- Worst-case impact: Set fee to zero for all pools, eliminating LP fee revenue. Install a fee module that returns 0 for all swaps. Install a MEV tax module that incorrectly classifies all flows as toxic, penalizing retail traders.

PoolFactory Discount Registry Manager (`discountRegistryManager`)

- Trust level: Partially trusted.
- Permissions: Set the `DiscountRegistry` contract address and transfer the `discountRegistryManager` role.
- Worst-case impact: Point to a malicious `DiscountRegistry` that returns the maximum discount for all callers, eliminating swap fees for all traders. The `DynamicSwapFeeHook` reads the discount registry without try/catch (no revert path expected), so a reverting registry would break CL swap fee computation.

PoolFactory Pool Tape Manager (`poolTapeManager`)

- Trust level: Partially trusted.
- Permissions: Set the `PoolTape` contract address and gas limit. Transfer the `poolTapeManager` role.
- Assumptions: Pool tape calls from pools are wrapped in `excessivelySafeCall`. Failures do not revert swaps.
- Worst-case impact: Redirect pool tape recording to a malicious contract. As tape calls use `excessivelySafeCall`, the worst outcome is corrupted analytics data visible to external tape consumers. Pool invariants and swaps are unaffected.

PoolTape and CLPoolTape Owner

- Trust level: Partially trusted.
- Permissions: Set the global default cadence interval and per-pool cadence. Set the allowed caller per pool (which address may call `record`). Transfer ownership via `Ownable2Step`.
- Assumptions: The allowed caller for each pool is the pool's registered factory.
- Worst-case impact: Set the allowed caller to a malicious address, enabling injection of false cumulative fee, volume, and MEV data. Downstream consumers of tape data (e.g., fee modules, analytics) may receive incorrect inputs. Swap execution itself is unaffected.

CLPoolTape Elastic Fee Module (`elasticFeeModule` on `CLPoolTape`)

- Trust level: Partially trusted.
- Permissions: Call `CLPoolTape.recordVolatility` to write the `volatilityCorrob` and `nOver` fields on the latest observation.
- Worst-case impact: Inject false volatility data into the tape, which may affect consumers reading corroboration values. No impact on swap execution or fund safety.

DynamicSwapFeeHook Swap Fee Manager (`swapFeeManager` from CL factory)

- Trust level: Partially trusted.
- Permissions: Set default scaling factor, default fee cap, per-pool custom fee, per-pool fee cap, per-pool scaling factor, per-pool initial fee, the global `secondsAgo` oracle window, and the `IMevTaxModule` address. Also controls bulk fee and fee-cap updates across all CL pools.
- Worst-case impact: Set fees to the maximum (50%) for all CL pools, extracting maximum fees from traders. Set fees to zero for all pools, eliminating fee revenue for LPs. Configure an incorrect MEV tax module, misclassifying all traffic as MEV and charging full MEV tax on every swap.

LP Providers (End Users)

- Trust level: Untrusted.
- Permissions: Deposit and withdraw LP tokens in `V2Gauge`. Mint and burn LP tokens in pools via `Pool.mint` and `Pool.burn`. Claim accrued emissions via `Gauge.claimEmissions`. Claim pool trading fees from `Pool.claimFees`. Grant and revoke claim and withdrawal approvals for operators via `Gauge.approveForClaim`, `Gauge.approve`, and `Gauge.setApprovalForAll`.
- Worst-case impact: Griefing through `depositFor()` (resetting the early-unstake penalty timer for another staker) is blocked for gauges with an active penalty unless the caller is an approved Metarouter. No mechanism for one LP to steal another's deposits or emissions.

veNFT Holders / Voters (End Users)

- Trust level: Untrusted.
- Permissions: Claim trading fees from `VotingRewardsManager.claimFees`. Claim incentive rewards from `VotingRewardsManager.claimIncentives`. Advance global reward checkpoints via `advanceGlobalPoints`.
- Worst-case impact: A voter calling `advanceGlobalPoints` or `claimFees` at an adversarial time cannot steal other voters' share. Reward computation is based on verifiable checkpoint history.

Incentive Providers (End Users)

- Trust level: Untrusted.
- Permissions: Create incentive streaming programs in `VotingRewardsManager.createIncentiveProgram` for any token listed in the `TokenRegistry`, with a minimum duration of 7 days and a minimum amount equal to the duration. Sweep unclaimed tokens from expired programs back to the creator via `sweep`.
- Worst-case impact: A program creator can only sweep tokens proportional to zero-supply seconds in the program window. No other user's incentive or fee entitlement is affected.

Metarouter Users (End Users)

- Trust level: Untrusted.
- Permissions: Execute composable command batches via `Metarouter.execute`, including CL and V2 swaps, liquidity management, gauge staking/unstaking, incentive staking, fee claims, ETH wrapping/unwrapping, NFT transfers, cross-chain bridging, and (on root only) VotingEscrow staking and relay deposits.

- Assumptions: Pool routes are validated against `FactoryRegistry`. Only registered pools are accepted. ERC721 custody uses a transient expected-sender slot, preventing unsolicited NFT injection.
- Worst-case impact: A user executing a malformed batch loses only their own tokens, which are returned at batch closure if unused. The router enforces that no ERC20 or NFT is left in custody at batch end.

Gauge Operators / Claim Delegates (End Users)

- Trust level: Untrusted.
- Permissions: Withdraw LP tokens and claim emissions on behalf of an account holder in `V2Gauge`, subject to granted allowances or approvals. For V2 gauges using `approve`, allowances are decremented on use. `setApprovalForAll` grants unlimited allowance.
- Worst-case impact: A claim delegate can only act up to the allowance or approval granted by the position owner. No escalation beyond the granted scope is possible.

4.2 External Dependencies

LeafVoter

- Role in system: Authorizes operators for `VotingRewardsManager._checkAuthorized`. Provides `settleGauge` and cumulative reward share accounting for `V2Gauge`. Calls `VotingRewardsManager.checkpoint` on each vote. Mints and forfeits emissions. Hosts all `Roles.sol` roles used by `GaugeManager`, `DefaultGaugeCreationModule`, `GovernanceCreationModule`, and `FactoryRegistry`. `LeafVoter` is the canonical trust root for the entire in-scope system.
- Assumption: Operates correctly and is not compromised.
- Worst-case impact if compromised: `settleGauge` returning incorrect cumulative reward values would allow arbitrary emission inflation or deflation per gauge. `mintEmissions` being callable directly would allow minting tokens to arbitrary addresses. Role assignments would allow any in-scope privileged action.

CL Factory and CL Pools

- Role in system: `DynamicSwapFeeHook` reads `isPool`, `swapFeeManager`, `discountRegistry`, `clPoolTape`, and `tickSpacingToFee` from the CL factory. CL pools call `beforeSwap` and `afterSwap` on the hook during swaps. `CLPoolTape` reads `secondsPerLiquidityCumulativeX128` from CL pools.
- Assumption: The CL factory and pools are uncompromised and only registered CL pools call hook functions.
- Worst-case impact if compromised: A malicious CL pool calling `beforeSwap` can trigger incorrect fee storage. A compromised `swapFeeManager` pointer enables all swap fee manager actions described above.

CL LP Staker (`SlipstreamStaker`)

- Role in system: Used by `Metarouter` for LP incentive staking and unstaking commands. The Metarouter checks that the LP staker's `nonfungiblePositionManager` matches the configured position manager at construction.
- Worst-case impact if compromised: A malicious staker could withhold or return incorrect NFT IDs, causing batch failures or leaving NFTs trapped in the router, triggering a revert at batch closure.

NonfungiblePositionManager

- Role in system: Used by `Metarouter` for all CL position operations (mint, increase, decrease, collect, burn). Also the sole accepted NFT sender in `onERC721Received`.
- Worst-case impact if compromised: A malicious position manager could send unexpected token IDs or return incorrect owners, bypassing custody checks or causing batch-closure reverts.

Hyperlane Mailbox

- Role in system: The source-chain Mailbox dispatches messages for `BRIDGE_TOKEN` and `EXECUTE_CROSS_CHAIN` commands. The destination-chain Mailbox delivers them to the bridge contract or ICA router after ISM verification.
- Assumptions: The Mailbox correctly enforces replay protection by tracking delivered message IDs. Each message is delivered at most once. The Mailbox contract is not upgradeable by a compromised owner during the audit period.
- Worst-case impact if compromised: Replay of delivered messages could trigger multiple bridge mints or ICA executions from a single burn or dispatch.

Hyperlane ISM (Interchain Security Module)

- Role in system: Called by the destination Mailbox before delivering each message. It verifies that the message was genuinely dispatched on the claimed source chain.
- Assumptions: The ISM configured for HypXERC20 and the ICA Router is correctly set up with an honest validator majority. The ISM owner does not replace it with a permissive or no-op module.
- Worst-case impact if compromised: Forged messages can trigger bridge mints without corresponding burns, or execute arbitrary calls on a user's ICA.

Hyperlane Relayer

- Role in system: An off-chain service that calls `Mailbox.process()` on the destination chain to complete message delivery. For commit-reveal ICA flows, it fetches pre-images from the off-chain gateway and submits them as metadata.
- Assumptions: At least one honest relayer is available for each relevant chain pair. Relayers cannot forge messages or bypass ISM verification. Pre-image commitments for commit-reveal flows use fresh random salts.
- Worst-case impact if unavailable: Cross-chain message delivery stalls indefinitely. Bridge burns may not result in destination mints until a relayer becomes available. Anyone can run a permissionless relayer to rescue stuck messages.

Hyperlane xERC20 Bridge (HypXERC20)

- Role in system: Used by the `BRIDGE_TOKEN` command to burn xERC20 tokens on the source chain and mint them on the destination chain.
- Assumptions: The ISM securing the bridge correctly validates message origin. The xERC20 token owner does not revoke or reduce the bridge's mint or burn rate limits to zero. No other bridge to the same token exhausts the rate limit ahead of a pending Hyperlane delivery.
- Worst-case impact if compromised: Tokens burned on source may not be minted on destination, or mints may occur without corresponding burns.

Interchain Account Router (ICA Router)

- Role in system: Used by `Metarouter` for `EXECUTE_CROSS_CHAIN` commands to dispatch cross-chain calls via a commit-reveal scheme.
- Assumptions: ICA addresses are derived deterministically from origin domain, owner address, remote router, and remote ISM. The ISM protecting inbound ICA messages correctly verifies the origin sender. Changing the ISM or remote router produces a different ICA address with no migration of existing state.
- Worst-case impact if compromised: Cross-chain commands could silently drop, reorder, or execute on behalf of an arbitrary origin address. In-scope contracts do not verify delivery. Users bear cross-chain execution risk.

RelayFactory (root chain only)

- Role in system: Used by `Metarouter.DEPOSIT_RELAY` to authenticate relay deposit targets.
- Worst-case impact if compromised: Deposits could be routed to unauthorized relay contracts. Only affects root-chain relay operations.

VotingEscrow / Staking Escrow (root chain only)

- Role in system: Used by `Metarouter` for `CREATE_STAKE` and `DEPOSIT_RELAY` commands to lock staking tokens and deposit to relay strategies.
- Worst-case impact if compromised: Misdirected stake creation or relay deposits. Staking token approvals made inside the batch are bounded to the amount transferred.

WETH

- Role in system: Used by `Metarouter` for `WRAP_ETH` and `UNWRAP_WETH` commands. Used by `VotingRewardsManager` to auto-unwrap native reward tokens on claim.
- Assumption: WETH correctly wraps and unwraps native ETH and is the canonical wrapped native token on the chain.
- Worst-case impact if compromised: Incorrect ETH accounting in batch execution. A malicious WETH unwrapping on a `VotingRewardsManager` claim could reenter, mitigated by the `ReentrancyGuardTransient` on `claimFees`.

Pool Fee Tokens (ERC20)

- Role in system: Fee tokens are held in `PoolFees` and transferred to `VotingRewardsManager` on `collectFees`. Incentive tokens are held in `VotingRewardsManager` until claimed.
- Assumption: Fee tokens and incentive tokens behave as standard ERC20 tokens.
- Worst-case impact if non-standard: Fee-on-transfer tokens would cause accounting mismatches between `lastPendingFees` and actual balances. Rebasing tokens would silently change claimable balances over time. Neither case is handled in-scope.

Referral Addresses

- Role in system: Referral addresses receive a share of gauge emissions when `REFERRAL_ADMIN_ROLE` or the gauge creator has configured a referral. Emissions are minted directly to the referral address by `LeafVoter`.
- Assumption: Referral addresses are set deliberately by the `REFERRAL_ADMIN_ROLE` or by gauge creators at creation time.
- Worst-case impact if compromised: Referral emissions are minted to the compromised address. LP staker emissions are unaffected as they are computed net of the referral split.

4.3 External Dependencies

Gauge staking token

- Assumption: the staking token of a fee-bearing gauge (`isPool == true`) is a Velodrome pool (`StablePool` or `VolatilePool`). These inherit OpenZeppelin `ERC20` without overriding the transfer path, so transferring the staking token never hands control to the recipient.
- Worst-case impact: a re-entrant staking token lets a withdrawer take control inside `Gauge.withdraw()` while the gauge's reentrancy lock is held, and re-enter any `nonReentrant` gauge function to force it to revert.

Incentive and reward tokens

- Assumption: every token listed in the `TokenRegistry` transfers the exact amount requested. Fee-on-transfer, deflationary and negatively rebasing tokens are out of scope and must not be listed.
- Worst-case impact: `IncentiveStreaming.createIncentiveProgram()` derives `rate`, `amount` and `remainingAmount` from the requested figure and only then pulls the tokens, without reading its own balance, so a token that delivers less leaves a program credited for tokens the contract does not hold. Claims are paid at the inflated rate until the balance runs out, so the last claimants and the creator's `sweep()` revert on transfer, and the shortfall lands on whoever settles last rather than on the creator.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 A Non-Standard Pool Token Can Block Gauge Fee Collection

System Consideration	Version 1
<code>V2Gauge.collectFees()</code> forwards to <code>IPool.claimFees()</code>, which transfers the pool's <code>token0</code> and <code>token1</code> to the <code>VotingRewardsManager</code>. The pool's tokens therefore govern whether fee collection succeeds. A token that reverts, pauses, or blocklists the manager makes <code>claimFees()</code> revert, as does one that hands control to a third party during the transfer, since re-entering the gauge while its <code>nonReentrant</code> lock is held reverts. A fee-on-transfer token instead breaks the accounting, because the manager credits the amount <code>claimFees()</code> reports while receiving less. The consequence depends on the path. <code>VotingRewardsManager._claimFees()</code> calls <code>collectFees()</code> directly, so a revert propagates and blocks fee claims for every voter on that gauge. <code>flushFees()</code> wraps the same call in <code>excessivelySafeCall</code> with a <code>COLLECT_FEES_GAS_LIMIT</code> of <code>1_000_000</code>, so the revert is swallowed as <code>FeeCollectionFailed</code> and the gauge is suspended with its fees stranded. <code>GaugeFactory.createGauge()</code> checks neither token, so whoever authorizes a gauge has to screen the pool's tokens. A separate reentrancy-lock variant is described in Gauge Fee Collection Reverts if the Reentrancy Lock Is Held.	

SC2 depositFor() Lets Anyone Raise an Account's Staked Balance

System Consideration	Version 1
<code>depositFor()</code> on the gauge (<code>Gauge.depositFor()</code>, implemented in <code>V2Gauge</code>) lets any caller stake LP tokens on behalf of an arbitrary <code>_owner</code>. The tokens are pulled from the caller, but the staked balance is credited to <code>_owner</code>, so an account's staked balance can increase without any action by the owner via donation. Integrations that assume an account's staked balance changes only through that account's own actions, or that it mirrors an external balance they track, can be desynchronized by an unsolicited <code>depositFor()</code>. A protocol building on the gauge cannot treat a staked balance as reflecting only its own deposits.	

SC3 Operator Authorization Assumes One Controller Across Chains

System Consideration	Version 1
<code>Voter.setOperator()</code> on the root chain proves control of the veNFT through <code>VotingEscrow.isAuthorized()</code>, then dispatches an arbitrary operator address to the chosen leaf chain, where <code>LeafVoter.setOperator()</code> records it under <code>onlyMessageOrchestrator</code>. The leaf then authorizes purely by <code>msg.sender == tokenState[_tokenId].operator</code>, with no proof binding that address to the root-chain owner. Authorization therefore assumes the operator address has the same controller on every chain. That holds for EOAs, whose key is fixed across chains, but not for contract wallets, which can be undeployed or controlled by a different party at the same address on a leaf chain. Whoever controls the address there can act as the	

operator. A user who sets a contract wallet as operator must control that address on every chain where the protocol is deployed.

The per-gauge `approveForClaim()` approval is local to each chain and is not propagated. `LeafVoter.claimFees()` and `claimIncentives()` are still stubs at this version, so the same-address assumption also constrains that pending functionality.

SC4 Priority of Caller Discounts

System Consideration	Version 2
----------------------	-----------

`DiscountRegistry` is a standalone contract reached through the pool factory rather than state held by the fee module, so one grant applies wherever it is queried. `IDiscountRegistry` documents `getDiscount()` as resolving in strict priority order

1. caller's per-pool discount
2. caller's chain-wide discount
3. tx's origin per-pool discount
4. tx's origin chain-wide discount.

and returning the discount given at the first match. `DynamicSwapFeeHook._applyDiscount()` passes the pool-level `msg.sender` as that caller, so if any router is given a small discount, users with a larger personal discount must avoid that router and swap directly with the pool instead.

Similarly, a user holding a personal discount must ensure that nobody else can spend it. Because the lookup falls back to `tx.origin`, any transaction that hands control to an untrusted contract, for example a swap routed through a malicious token pair, lets that contract run a swap of its own at the origin's discount.

`MAX_DISCOUNT` is `800_000` against a pips denominator of `1e6`, so at most 80 percent of the fee can be taken this way.

SC5 Security Considerations for Gauge Integrators

System Consideration	Version 1
----------------------	-----------

Integrating contracts that stake into a `V2Gauge` should consider the following properties of the early-unstake penalty:

1. **The penalty applies to all accrued emissions:** the gauge keeps a single `depositBlock` per account and rewrites it on every deposit. A top-up therefore restarts the window on the whole position, and the penalty is charged on everything accrued, including emissions earned long before it. At the factory defaults of `DEFAULT_PENALTY_RATE = MAX_PIPS` (100%) and `DEFAULT_MIN_STAKE_BLOCKS = 5`, a top-up followed by a claim or withdrawal within 5 blocks forfeits the entire amount.
2. **A penalty applied is not exposed to the caller:** `claimEmissions()` and `withdraw()` return no amount and do not revert, and only the `EarlyWithdrawPenalty` event records the loss. The view function `earned()` is no help either, since it already reports emissions net of the penalty.
3. **`earned()` can fall without the staker doing anything:** the penalty is evaluated when the view runs, against the config read fresh from the factory, and `PENALTY_ADMIN_ROLE` can raise `minStakeBlocks` or `penaltyRate` at any time for positions already staked. At the default rate of 100 percent a config change alone takes `earned()` from its full value to zero, so the view is not monotonic and cannot be cached.

An integration must enforce its own safety check by querying `depositBlock` and `minStakeBlocks` before claiming, and by checking the tokens received back.

SC6 Security Considerations for Keying on Pool Name or Symbol

System Consideration

Version 2

In `Pool`, `name()` and `symbol()` are derived from the underlying token symbols at initialization and can be overwritten at any time by the factory's `poolAdmin`. No contract in the protocol consumes either string, but some projects derive the salt for a deterministic deployment from token symbols rather than addresses, so the same salt is derived for tokens with the same symbol across different chains.

```
salt = keccak256(abi.encodePacked(IERC20(token0).symbol(), "/", IERC20(token1).symbol()))
```

MetaDEX Application Layer's `Pool` tokens are incompatible with that approach, and any change in name or symbol might break such integrations.

SC7 Security Considerations for Pool Depositors

System Consideration

Version 2

`Pool.mint` ensures that at least one LP token is getting minted.

```
if (_totalSupply == 0) {
    liquidity = Math.sqrt(_amount0 * _amount1) - _MINIMUM_LIQUIDITY;
    _mint(address(1), _MINIMUM_LIQUIDITY);
    _mintValidation(_amount0, _amount1);
    if (liquidity < _MINIMUM_LIQUIDITY) revert InsufficientLiquidityMinted();
} else {
    liquidity = Math.min((_amount0 * _totalSupply) / _reserve0, (_amount1 * _totalSupply) /
        _reserve1);
    if (liquidity == 0) revert InsufficientLiquidityMinted();
}
```

The first depositor locks `_MINIMUM_LIQUIDITY` (1000 shares) to `address(1)` and requires the depositor to receive at least 1000 more, so supply starts at 2000. The second branch checks only that the floored `min()` is non-zero. Anyone can therefore seed a pool at that minimum, and then inflate the value of LP tokens by transferring tokens and then calling the permissionless `sync()` to inflate the share price.

A deposit worth just under two shares could end up just minting one share, so they could forfeit up to half of their deposit. Half of the initial donation is given to the locked shares, so this attack is only worth it when a large number of deposits is rounded down.

Still, callers are expected to implement their own slippage protection. Depositors should quote the shares they expect and reject a mint that returns materially fewer.

SC8 Security Considerations for Pool Tape Consumers

System Consideration

Version 2

`observe()` on a pool tape has two unexpected behaviors:

- A read against an unwritten buffer returns an all-zero observation rather than reverting
- A target older than the oldest stored observation is answered with the oldest observation rather than rejected

Consumers must read `observationBuffers()` for the index and both cardinalities, and `accumulators()` for the last swap timestamp, before trusting any observation. An all-zero observation signals absence of data, and every returned `blockTimestamp` must be checked against the target that was asked for. No contract in the reviewed code reads this function, so the exposure is confined to external integrators.

SC9 Usage of Allow Revert Flag in Metarouter

System Consideration	Version 2
----------------------	-----------

Any user of the Metarouter should be careful when crafting commands that use the `allow-revert` flag.

A number of MetaDEX Application Layer contracts guard their entry points with a `nonReentrant` modifier, among them `Pool`, `Gauge`, and `VotingRewardsManager`. Delivering a Hyperlane message is permissionless, so whoever delivers it can enter one of those contracts first and then execute the message in a callback, so that any command touching the same contract reverts.

If that command carries the `allow-revert` flag, the batch still succeeds, but every later command runs against the state the skip left behind. Users should set the flag only where omitting the command is acceptable in every case, not where failure merely looks unlikely, and never on a command whose result a later command depends on. If any commands are dependent on each other, they should be left unflagged, so the batch aborts and the message can be relayed a second time.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

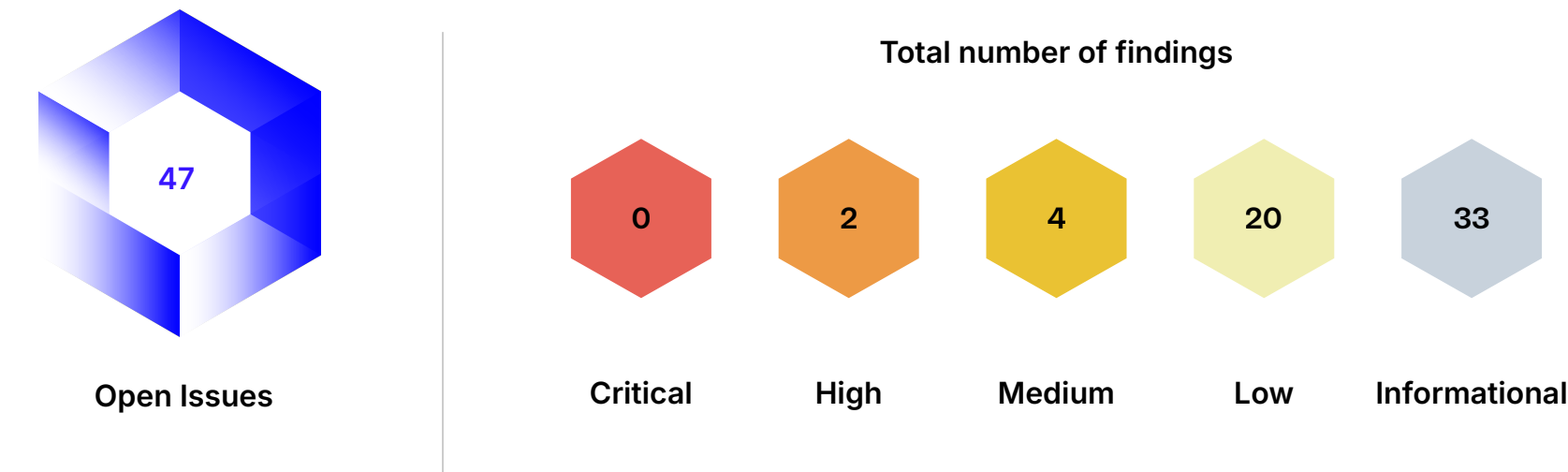
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the [Terminology](#) section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

High	#001	A Failed Checkpoint Keeps a veNFT's Weight on Two Gauges	Code Corrected	✓
High	#002	Attacker-Caused Missing Gauge Rate Windows	Code Corrected	✓
Medium	#003	Chain Suspension Locks Staked LP Tokens	Open	○
Medium	#004	Hardcoded Gas Stipends Cannot Adapt to Gas Repricing	Open	○
Medium	#005	Insufficient Gas Griefing to Trigger the Else Branch	Open	○
Medium	#006	Malicious Relayer Can Forfeit All Accrued Emissions	Open	○
Low	#007	A Dust Swap at the Top of the Block Avoids a High Initial Fee	Open	○
Low	#008	A Size-Dependent Fee Breaks the Exact-Output Quote	Open	○
Low	#009	A Small Swap Pays No Fee	Open	○
Low	#010	Adding Liquidity Through the Router Can Lose Half the Deposit	Open	○
Low	#011	Anyone Can Distort the Staked Liquidity Value in the Pool Tape	Open	○
Low	#012	Assigning the Cap Operator Role Blocks Gauge Deactivation	Open	○
Low	#013	Changing the Default Cap Alters Already Accrued Emissions	Open	○
Low	#014	createIncentiveProgram Pulls More than It Distributes	Code Corrected	✓
Low	#015	Fees and Incentives Lock Once Checkpoint History Falls Behind	Open	○
Low	#016	Gauge Fee Collection Reverts if the Reentrancy Lock Is Held	Open	○
Low	#017	Inaccurate NotifyRewardRate Event Data	Code Corrected	✓
Low	#018	Incoming Bridge Transfer Corrupts Swap's Slippage Check	Open	○
Low	#019	Missing Gauge Linkage Check in createGauge	Open	○
Low	#020	Missing Underflow Checks in _earnedIncentiveOpenInterval	Open	○
Low	#021	Penalty Timers Can Be Reset Before Activation	Open	○



Low	#022	Racing Gauge Deployment to Set the Initial Referral Recipient	Open	
Low	#023	Router Uses Unbounded Claim Entrypoint	Open	
Low	#024	Swaps and Quotes Fail on a Stable Pool with Large Reserves	Open	
Low	#025	Unbounded MEV Tax Can Push the Fee Past the Traded Amount	Open	
Low	#026	Vote on a Dormant Rewards Contract Corrupts Voting Supply	Open	
Info	#027	A Third Party Can Make an LP Position Burn Fail	Open	
Info	#028	Checkpoint Index Helper Naming Omits Rounding Direction	Open	
Info	#029	Compiler Version Affected by Known Bug	Open	
Info	#030	Cross-Chain Commands Do Not Check Most Hyperlane Inputs	Open	
Info	#031	Duplicated Constant and Magic Values in the Gauge Contracts	Open	
Info	#032	Elapsed Time Is Recomputed in _createGlobalRewardPoints	Code Corrected	
Info	#033	Exact-Output V2 Routes Do Not Verify the Amount Received	Open	
Info	#034	Fee Caching Assumes Block.number Counts the Chain's Blocks	Open	
Info	#035	Fee Collection Is Gated on the veNFT's Current Balance	Open	
Info	#036	Fee Values Are Indexed in the Dynamic Fee Hook Events	Open	
Info	#037	Gas Optimizations in Pool and PoolOracle	Code Corrected	
Info	#038	Gas Optimizations in VotingRewardsManager Reward Math	Open	
Info	#039	getAmountOut Quotes a Fee the Caller May Not Receive	Open	
Info	#040	Hyperlane Dependency Pinned to Outdated Version	Open	
Info	#041	Inaccurate Comment on the Emission Cap	Open	
Info	#042	Incomplete Empty-Window Clause	Code Corrected	
Info	#043	Low-Level Storage Access in _createGlobalRewardPoints()	Open	
Info	#044	Misleading Variable Name Xy in _getAmountOut()	Code Corrected	
Info	#045	Missing Bounds Check on Program ID for Incentives	Open	
Info	#046	Missing Event Emission in _claimFees()	Code Corrected	
Info	#047	Native Asset with an ERC-20 Entry Point Makes a Batch Revert	Open	
Info	#048	No Check That Gauge Address Is as Expected	Open	
Info	#049	Oracle Readiness Check Is Stricter than the Revert It Guards	Open	
Info	#050	Referral Config Is Read as a Flattened Tuple	Open	
Info	#051	Single and Bulk Fee Setters Duplicate Validation	Open	
Info	#052	The Fee Resolution Order Is Undocumented	Open	
Info	#053	The Maximum Fee a Fee Module May Return Is Undocumented	Open	
Info	#054	The per-Interval Swap Count Saturates	Open	
Info	#055	Unchecked Return Data Length in PoolFactory.getFee()	Code Corrected	
Info	#056	Undocumented Viewer Deviations	Open	
Info	#057	Unreachable K == Xy Branch in _get_y()	Code Corrected	
Info	#058	Unused Data Argument in Checkpoint()	Open	
Info	#059	Unused Return Value of _notifyFeesAmount()	Code Corrected	

7.2 Descriptions

#001 A Failed Checkpoint Keeps a veNFT's Weight on Two Gauges

Security	High	Version 1	Code Corrected 
----------	------	-----------	--

`LeafMessageOrchestrator._applyTokenAllocation()` first applies a tokenId's new gauge allocation by calling into `LeafVoter.applyTokenAllocation()`, then forwards a reward checkpoint to `LeafVoter.checkpointReward()` that then forwards a checkpoint to each touched gauge via `VotingRewardsManager.checkpoint`. The two calls are not atomic. The second call is gas-capped and reverts are caught via `try/catch`. If the call fails, a retry ticket is stored while the allocation has already been written.

```
try LEAF_VOTER.checkpointReward{gas: _checkpointGasLimit}(_tokenId, _callParamsList[_i]) {}
catch {
  bytes memory _data = _callParamsList[_i].data;
  checkpointRetryHash[_tokenId][_gauge] = keccak256(abi.encode(_tokenId, _gauge, _data));
  emit CheckpointRewardFailed(_tokenId, _gauge, _data);
}
```

A checkpoint must be created whenever a tokenId's weight on a gauge changes by calling into `VotingRewardsManager.checkpoint()`. When the forward fails, the old `VotingRewardsManager` keeps the tokenId at its former weight while the new one counts the same power again. Both pay out pro rata against it. Both in `earnedFees()` as `_weight * _amount / _supply` and on the same basis in `earnedIncentives()`.

An attacker can make one forward fail while the rest succeed:

1. Hold a position on gauge A, then move it to gauge B with a root vote.
2. Deliver that vote message with a tight gas limit. Per EIP-150 the capped forward receives at most `gasleft() - gasleft() / 64`, so A's checkpoint runs out of gas while the `catch` and the remaining forwards complete. No malicious token is required.
3. Alternatively, hold A's guard across delivery. `checkpoint()` shares one reentrancy guard with `claimIncentives()`, `sweep()` and `claimFees()`, so an incentive token that makes a callback to its recipient can trigger the reentrancy guard that then makes the vote's checkpoint fail.
4. Claim fees and incentives on both A and B for the same tokenId.

There is no functionality to automatically fix this. A later vote clears the retry marker only for the gauges it touches. A permissionless function `retryCheckpointReward()` is available to re-execute the failed call. Anyone can call it, but somebody must first notice the `CheckpointRewardFailed` event and rebuild the payload, so correct reward accounting depends on off-chain upkeep instead of holding by construction.

Until then the tokenId draws `_weight / _supply` of every fee and incentive distribution on gauge A on top of its full entitlement on B, and the same steps repeat across gauges to hold weight on many at once.

Note on the audit process: This issue was reported to ChainSecurity by MetaDEX Application Layer while the audit was in progress. Its root cause has been discovered internally already, though the severity had not been fully assessed yet.

Code Corrected

The gas-capped forward was removed with the voting redesign. `LeafVoter` now drives the per-gauge checkpoints itself in `_driveCheckpoints()`, without a gas cap and without `try / catch`, so a reverting reward contract fails the whole allocation instead of leaving the allocation written and the checkpoint pending. The retry ticket and `retryCheckpointReward()` no longer exist.

#002 Attacker-Caused Missing Gauge Rate Windows

Design

High

Version 1

Code Corrected 

Gauge Rate Notification Failure

When the `LeafMessageOrchestrator` processes a `Vote` message, it calls the leaf voter multiple times. First, `LEAF_VOTER.applyChainState` is called, which sets the chain-wide `lastSettlement` field. Then, it calls `LEAF_VOTER.applyTokenAllocation`, which ultimately runs `_processGauge` for each affected gauge. Finally, this means that each of the `GaugeState.lastSettlement` values will now be set to the same value as the chain-wide `lastSettlement`.

The next step is for the `LeafMessageOrchestrator` to call `LEAF_VOTER.notifyGaugeRate` for each of the gauges. These calls are all wrapped in a `try / catch` block, meaning that any failure will only result in the emission of a `NotifyGaugeRateFailed` event, but will not propagate the revert. The `Vote` message as a whole will still successfully complete, meaning the nonce will be consumed and no retry is possible. Unlike the reward checkpoints, there is no retry mechanism for the gauge rate notifications.

A comment in the code states: "A failed notify is swallowed because the permissionless `LeafVoter.settleGauge` remediates it." However, this is **incorrect**. As previously mentioned, the `LeafVoter` has already updated the gauge-specific `lastSettlement` value, meaning the next call to `settleGauge` will not cover the skipped window. Instead, the gauge will simply extend the previous rate across the intervening period, resulting either in over- or under-distribution of rewards. Among other issues, this breaks the internal accounting of rewards in the `LeafVoter`.

Attack Feasibility

There are several ways in which the call to `LEAF_VOTER.notifyGaugeRate` may fail. Firstly, a misconfigured gas limit may result in the call running out of gas. This could, for example, happen when gas pricing changes in the upcoming Glamsterdam hard-fork, or other future gas pricing changes.

Secondly, the subsequent call to `Gauge.notifyRewardRate` may fail. Since anyone can relay cross-chain messages, an attacker may do so inside of a callback from the gauge, meaning that its reentrancy guard will reject the call to `notifyRewardRate`.

For a `V2Gauge`, such a callback could be triggered by a token with transfer hooks, such as an ERC-777 token. For any `CLGauge`, the transfer hook of the `ERC721` token is sufficient to cause such a failure. Thus, a malicious actor could cause any rate notification to a `CLGauge` to fail by front-running cross-chain messages.

Code Corrected

The push model was removed as part of the voting redesign. `notifyGaugeRate` no longer exists, and each gauge now pulls its cumulative reward share from `LeafVoter.settleGauge` when it advances rewards, so a failing notification can no longer skip a rate window.

#003 Chain Suspension Locks Staked LP Tokens

Design

Medium

Version 2

Open 

In `V2Gauge`, a withdrawal claims the depositor's pending emissions before returning the principal. `_claimEmissions()` calls `mintEmissions()` on the `LeafVoter` whenever the account has some claimable emissions.

`mintEmissions()` is protected by a `chainIsActive` modifier and reverts for any status other than `Active`.

```
enum ChainStatus {
    None,
    Active,
    Paused,
    Suspended
}
```

The `CHAIN_STATUS_ROLE` on the `LeafVoter` can change the chain state with `setChainStatus()`. The initial state is `None` and the chain will from here move into the `Active` state to start earning rewards. The `Paused` state is meant to be a temporary emergency measure. If a chain issue cannot be resolved the chain is moved into the `Suspended` state. Here the chain might stay indefinitely. Neither `Gauge` nor `V2Gauge` lets a user exit while skipping the claim, so `withdraw()` and `withdrawFrom()` revert, stranding the user's liquidity tokens indefinitely.

Deposits, however, remain enabled by the gauge while the chain is suspended, accepting principal that it cannot return. The only exit left is for the staker to slash itself, which is possible in principle as long as the penalty rate is 100 percent. With all emissions forfeited, the call to `mintEmissions()` is skipped. However, when the rate is below 100 percent, or if the user (i.e. an integrating smart contract) guards against self-slashing, the liquidity remains permanently stranded.

#004 Hardcoded Gas Stipends Cannot Adapt to Gas Repricing

Security

Medium

Version 1

Open 

Several gas-limited external calls forward a constant gas limit to the callee. `PoolFactory.recordPoolTape()` forwards `300_000` to the pool tape, `PoolFactory.getFee()` forwards `500_000` to the fee module, and `VotingRewardsManager.flushFees()` forwards `COLLECT_FEES_GAS_LIMIT` (`1_000_000`, a public constant) to `collectFees()`. `PoolFactory.createPool()` forwards `gasleft()` instead, but hardcodes the amount of work through `_DEFAULT_POOL_TAPE_CARDINALITY`, an internal constant fixed at `100` that already carries a `// TODO: Revisit this value` note. None of these values has a setter.

```
_poolTape.excessivelySafeCall(300_000, 0, 0, abi.encodeCall(IPoolTape.record, (msg.sender,
_data)));
```

In each of these instances an out-of-gas failure in the callee is swallowed silently rather than reverting, as described in [Insufficient Gas Griefing to Trigger the Else Branch](#). Some stipends are already tight against current gas costs. `recordPoolTape`'s `300_000` must cover `record()`, which writes the full accumulator, a 5-slot struct, and up to 5 of those slots go from zero to nonzero on the first record. If the observation buffer was never initialized, that record also runs `_initializePoolBuffer()`, adding 2 more, for up to 7 zero-to-nonzero writes in a single call.

`createPool()`'s buffer pre-warm is the largest of these. The loop in the pool tape writes 5 slots per observation and runs 99 iterations, so reaching a cardinality of `100` needs 495 zero-to-nonzero writes, roughly 9.9 million gas at today's cost. The impact of a failed pre-warm is bounded, since `increaseObservationCardinalityNext()` is permissionless and grows the buffer from wherever it currently stands, so the cardinality can always be raised afterwards, in chunks if one transaction cannot hold it.

The Glamsterdam upgrade raises the cost of zero-to-nonzero storage writes, so stipends sized against today's costs can fall short afterward. The cost of a zero-to-nonzero storage write is expected to increase from 20,000 to 100,000 gas. Note that Glamsterdam is also expected to add a separate `state_gas_reservoir` for storage writes, which is populated by setting a gas limit above the per-transaction cap of roughly 16 million gas, so the affected calls would not be permanently bricked.

They would however require the caller to set a very large initial gas limit to succeed, with potentially problematic UX consequences, and insufficient gas grieving attacks would become trivially achievable. At the same repricing, that pre-warm loop costs roughly 49.5 million gas, about three times the per-transaction cap, so `createPool()` could no longer warm a buffer to `100` in one transaction and every new pool would need its cardinality raised out of band.

Because the cardinality is hardcoded, it cannot be lowered to fit either. Further, because the stipends are hardcoded, the protocol cannot raise them in response to any later repricing of state costs, which is always possible and could lead to more critical flaws.

Changes in **Version 2**

- The pool tape and fee module stipends are no longer hardcoded.

#005 Insufficient Gas Grieving to Trigger the Else Branch

Security	Medium	Version 1	Open
----------	--------	-----------	------

Several contracts wrap an external call in `excessivelySafeCall` or `excessivelySafeStaticCall` to handle reverts. The library functions take a gas argument, but this is only an upper bound on the gas passed to the callee and gives no guarantee about the amount actually forwarded. Further, these libraries cannot distinguish a genuine revert from an out-of-gas failure, as this is not exposed by the EVM.

```
(bool _success, bytes memory _data) = feeModule.excessivelySafeStaticCall(
    500_000, 32, abi.encodeCall(IFeeModule.getFee, (_pool, _caller, _amount0In, _amount1In,
        _reserve0, _reserve1))
);
if (_success && _data.length == 32) { /* use the custom fee */ } // else fall back to
defaultFee
```

The module itself can make use of at most `min(requested, gasleft() - gasleft() / 64)`, per EIP-150. The gas available therefore depends on how much gas is remaining at the time of the call, and as such on the initially specified transaction gas limit, which is ultimately controlled by the original sender of the transaction.

A malicious caller could pick a top-level gas limit such that `gasleft()` at the call site is below the requested stipend. The module then runs out of gas if the forwarded amount is insufficient, while the one sixty-fourth retained by the calling frame is still enough to complete the outer logic.

The caller is left with a free choice between letting the call succeed and forcing the fallback, and takes whichever is more favorable. There are multiple sites where this is possible:

1. `PoolFactory.getBaseFee()` : falls back from a custom fee to the default `fee`.
2. `PoolFactory.getFee()` : falls back from the MEV tax module to a zero tax and a non-toxic classification.
3. `PoolFactory.recordPoolTape()` : drops the oracle tape record.
4. `PoolFactory.createPool()` : skips the initial observation-cardinality bump.
5. `VotingRewardsManager.flushFees()` : leaves gauge fees uncollected on suspension, surfaced only as `FeeCollectionFailed`.

In practice, whether this is exploitable depends heavily on the module's actual gas use and on the gas cost of the execution thereafter, which still has to succeed. Both are notoriously hard to calculate, though. Where it is feasible, the caller chooses which fee swap pricing is based on, strands a gauge's pending fees, drops oracle tape records, which carries second-order effects if observations go unrecorded for a long time, or is quoted a fee below the one the swap will charge.

Changes in Version 2

Sites 1, 2 and 3 now check the gas left before the call. Site 4 no longer makes a guarded call. Additional `try/catch` have been added passing 63/64 of the gas left:

6. `DynamicSwapFeeHook`'s fee resolution: falls back from the module's MEV tax to a zero tax and a non-toxic classification.
7. `DynamicSwapFeeHook._getDynamicFee()`: falls back to using the base fee.
8. `DynamicSwapFeeHook.afterSwap()`: drops the pool tape record.
9. `FactoryRegistry.emissionCap()`: falls back from the gauge factory's cap to zero.
10. `Metarouter`'s `ownerOf()` call: skips the custody check.

#006 Malicious Relayer Can Forfeit All Accrued Emissions

Design

Medium

Version 2

Open 

`V2Gauge.depositFor()` reverts when the penalty config is active, so that third parties cannot reset an account's early-unstake timer. However, an approved `Metarouter` is exempted from that rule:

```
if (
  _penaltyConfig.penaltyRate > 0 && _penaltyConfig.minStakeBlocks > 0
  && !FACTORY_REGISTRY.isMetaRouterApproved({_metaRouter: msg.sender})
) revert PenaltyActive();
```

This exemption is normally safe. The command `STAKE_GAUGE` deposits only for `MetarouterState.msgSender()`, the caller of the router, so no one can abuse the router's privileges to reset another account's timer. This exemption is in fact strictly needed to let users call the gauge through the router. It stays in the user's control when they deposit into the gauge, and whether that causes any slashing on a subsequent withdrawal.

However, messages that are executed cross-chain cannot have the time guarantees a user would normally have when executing on one chain. The expectation is that users send messages from one chain to the other chain via the router to their controlled interchain account, which then calls the router deployed on that other chain.

If they send a message to stake and then later one to unstake or claim, and the stake message has not been executed yet because of low gas, a malicious or misconfigured relayer can execute the deposit and the subsequent unstake or claim in short order.

The penalty then applies to everything the account has accrued, so emissions earned over months could be forfeited in full by a routed deposit that landed a block earlier, see [Security Considerations for Gauge Integrators](#).

Nothing in the router detects this: Neither `UNSTAKE_GAUGE` nor `CLAIM_GAUGE_REWARDS` reads `depositBlock`, `minStakeBlocks` or the effective penalty config, and neither compares the emission tokens received. The router caller could in theory add its own slippage check through the `BALANCE_CHECK` command, but that makes the router much harder to use, since it creates an implicit interdependence between commands where other commands such as swaps are self-sufficient and implement their own slippage checks.

#007 A Dust Swap at the Top of the Block Avoids a High Initial Fee

Design	Low	Version 2	Open
--------	-----	-----------	------

In `DynamicSwapFeeHook`, a pool with the initial fee enabled charges `initialFee` to the first swap of each block, and `applyDiscount(feeToStore)` on every subsequent swap in the same block.

The cost for the top of the block trade is a percentage fee, so if the former fee is larger than the latter (the expected use case), a trader wanting to avoid a high fee can put a dust swap at the top of the block and follow with a large one in the same transaction. The dust swap pays the initial fee on a small input, and fixes the block fee. The real swap pays the block fee.

#008 A Size-Dependent Fee Breaks the Exact-Output Quote

Design	Low	Version 2	Open
--------	-----	-----------	------

In `Pool`, `getAmountInWithTotalFee()` works out the input amount by proposing an amount, deriving a fee and then inverting and correcting, giving up with `QuoteFailed` after `_MAX_FEE_ITERATIONS` tries (set to 8). These are enough only if the fee does not change with trade size. However, nothing in the fee module interface prevents it from changing.

A fee that grows with trade size keeps the guesses climbing until the quote gives up, and since `Metarouter._v2SwapExactOut()` quotes every hop this way, one such pool breaks every exact-output route through it. A fee that shrinks with size does the opposite (settling too early on too high a number) so the caller overpays and the excess stays in the pool.

#009 A Small Swap Pays No Fee

Design	Low	Version 2	Open
--------	-----	-----------	------

In `Pool`, `swap()` computes the fee as `(amountIn * _fee) / _MAX_BPS` rounding down, so for any input below `_MAX_BPS / _fee` the fee rounds to zero. At 30 basis points the first 333 units traded are free, so the savings are more significant for high-value tokens with few decimals.

WBTC, as one popular example, has 8 decimals, and at 100,000 USD per bitcoin, one unit is worth 0.001 USD. A trade of 333 units, worth 0.33 USD, skips one unit of fee (0.001 USD). Splitting a larger trade into smaller trades is worth it as long as the gas costs of a swap are less than 0.001 USD. At the date of this report, gas costs on Base are of that order of magnitude.

#010 Adding Liquidity Through the Router Can Lose Half the Deposit

Design	Low	Version 2	Open
--------	-----	-----------	------

In `Metarouter`, the `ADD_LIQUIDITY` command bounds only the tokens it commits and ignores the LP tokens minted that are returned by `Pool.mint()`. A depositor can be rounded down by up to half the LP tokens in case the LP token price is inflated, see [Security Considerations for Pool Depositors](#).

Inflating the share price costs a griever more than any one victim loses, so it is only worth it if a large number of users deposit via the `Metarouter`. A contract integrating directly is more critical if it implements no additional

slippage check and has a permissionless deposit function that an attacker could trigger repeatedly with deposit amounts that achieve the maximum rounding error.

#011 Anyone Can Distort the Staked Liquidity Value in the Pool Tape

Security	Low	Version 1	Open 
----------	-----	-----------	--

`CLPoolTape.record()` advances `secondsPerStakedLiquidityCumulativeX128` using the `lastStakedLiquidity` left over from the previous record, then overwrites it with the staked liquidity at the time of the current swap.

```
if (_timeSinceLastSwap != 0 && _poolAccumulator.lastStakedLiquidity != 0) {
    _poolAccumulator.secondsPerStakedLiquidityCumulativeX128 += uint160(
        (_timeSinceLastSwap << 128) / _poolAccumulator.lastStakedLiquidity
    );
}
// ...
_poolAccumulator.lastStakedLiquidity = _data.stakedLiquidity;
```

Staking into a CL gauge or unstaking changes the pool's staked liquidity but writes no observation to the tape, so `lastStakedLiquidity` refreshes only on a swap. As such, an attacker could flash-loan both tokens, mint an in-range position, stake it in the gauge, swap one wei, then unstake. That leaves the divisor arbitrarily large until the next swap, which artificially lowers `secondsPerStakedLiquidityCumulativeX128`. The value has no on-chain consumer today, but the planned use of it to improve the efficiency of gauge rewards would inherit the exposure.

#012 Assigning the Cap Operator Role Blocks Gauge Deactivation

Design	Low	Version 2	Open 
--------	-----	-----------	--

In `GaugeFactory`, `setEmissionCap()` checks the roles of `msg.sender` in the following order: cap admin, then cap operator, then emergency council. The first matching branch is then taken to validate the input.

```
if (hasRole(CAP_ADMIN_ROLE, msg.sender)) {
    ...
} else if (hasRole(CAP_OPERATOR_ROLE, msg.sender)) {
    if (_emissionCaps[_gauge] == 0) revert NotAuthorized();
    ...
} else if (ILeafVoter(LEAF_VOTER).hasRole(Roles.EMERGENCY_COUNCIL_ROLE, msg.sender)) {
    if (_cap != 0) revert NotAuthorized();
}
```

A holder of the highly privileged emergency council role can be degraded to the cap operator role by granting them the `CAP_OPERATOR_ROLE`. The operator branch rejects a value of zero for `_cap`, which is the value required to deactivate a gauge. The `CAP_ADMIN_ROLE` of `GaugeFactory`, as the role admin of the `CAP_OPERATOR_ROLE`, can assign this role and therefore revoke a privilege of `LeafVoter`'s `EMERGENCY_COUNCIL_ROLE`, violating the role hierarchy.

#013 Changing the Default Cap Alters Already Accrued Emissions

Correctness	Low	Version 2	Open
-------------	-----	-----------	------

In `GaugeFactory`, `setEmissionCap()` settles the gauge before storing the new cap, so a new per-gauge cap only takes effect going forward. `setDefaultCap()` settles nothing. Gauges that point to the default cap via `DEFAULT_CAP_INDICATOR` are therefore walked over their whole unsettled period under the new cap the next time `LeafVoter._settleGauge()` is called. Raising the default cap retroactively increases their effective emissions and lowers the surplus. Lowering it does the reverse. The amount affected grows with the time elapsed since the gauge was last settled.

This breaks the property that pending rewards cannot change retroactively. The same applies to `Gauge.earned()`, whose returned value can jump in either direction once the default cap is changed.

#014 createIncentiveProgram Pulls More than It Distributes

Design	Low	Version 1	Code Corrected
--------	-----	-----------	----------------

`IncentiveStreaming.createIncentiveProgram()` pulls the full `_amount` but derives the per-second rate by floor division, `_rate = (_amount * _PRECISION) / _duration`. Both the claim path and `VotingRewardsManager.sweep()` are bounded by `_rate * seconds / _PRECISION`, so the remainder can never leave the contract.

Code Corrected

`createIncentiveProgram` now derives the program amount from the floored rate and pulls only that amount.

#015 Fees and Incentives Lock Once Checkpoint History Falls Behind

Design	Low	Version 2	Open
--------	-----	-----------	------

`VotingRewardsManager._createGlobalRewardPoints()` backfills missing weekly global checkpoints in a loop bounded by `MAX_CHECKPOINT_ITERATIONS = 520`, roughly 10 years. Each checkpoint writes up to eight cold storage slots (GlobalPoint: 2, FeeSnapshot: 4, SupplyAccumulator: 2), which totals $8 \times 22,100 = 176,800$ gas at current gas pricing.

So if more than 10 years of history had to be backfilled, `_createGlobalRewardPoints` would consume about 84 million gas, well beyond the per-transaction gas limit on Ethereum. A gap of that size is unlikely to arise in practice, but because `advanceGlobalPoints()` and the other entry points offer no way to request fewer iterations, the `VotingRewardsManager` would become permanently unusable.

A fixed bound is also fragile against future gas repricing: with a transaction gas reservoir set to the 60 million block gas limit on Ethereum mainnet, and Glamsterdam raising a cold storage write from 20,000 to 100,000 gas, the maximum number of iterations would be less than $60,000,000 \div 8 \div 100,000 = 75$.

#016 Gauge Fee Collection Reverts if the Reentrancy Lock Is Held

Security	Low	Version 1	Open 
----------	-----	-----------	--

The function `Gauge.collectFees()` is protected by a `nonReentrant` modifier. Similarly, the gauge withdrawal functions `withdraw()` and `withdrawFrom()` are protected by such a modifier, and both share the gauge's single reentrancy lock.

The V2 gauge's internal `_withdraw()` function that implements the withdrawal logic transfers the staking token to the caller. If this transfer gives the caller control over the execution (i.e. the staking token is re-entrant), they could re-enter into `collectFees()` to make this call fail.

More problematic, the Slipstream CL gauge's `_withdraw()` transfers the position NFT with `nft.safeTransferFrom` in its last line. That transfer invokes the `onERC721Received` hook on every recipient that has code deployed.

In most cases triggering a revert alone would be harmless. However, in the Velodrome protocol, governance sets the emission cap of a gauge to zero through `GaugeFactory.setEmissionCap()`, and a zero cap makes that function call into `VotingRewardsManager.flushFees()`, which reaches `collectFees()` through the `ExcessivelySafeCall` library.

That library performs a low-level call and simply returns the status of the call. `VotingRewardsManager` accepts a failed status and continues, because a revert should not block gauge deactivation.

So if an attacker can trigger this revert, then a gauge can be deactivated with its pending fees left unflushed. Those fees can no longer be collected while the cap is zero, which leads to an unfair distribution of the final fees owed to the voters.

Note that a zero-cap `setEmissionCap()` can only be called by addresses with the `CAP_ADMIN_ROLE` or by the emergency council. Normally these roles are fully trusted, making this attack unrealistic. However, if either role is held by a Safe or an OpenZeppelin Governor, the execution of approved calls is permissionless once the signature threshold or the vote quorum is reached, which is the default configuration for both. Any account can then execute an already-approved zero-cap `setEmissionCap()` from inside a gauge withdrawal callback and deactivate the gauge with its fees unflushed.

#017 Inaccurate NotifyRewardRate Event Data

Correctness	Low	Version 1	Code Corrected 
-------------	-----	-----------	--

`V2Gauge.notifyRewardRate()` emits the raw inputs `_rewardRate` and `_from`, and not necessarily the rate and range applied by the gauge. `V2Gauge._settleRewards()` folds the leftover emissions of a still-pending rate into the new `activeRate` and starts the window at `min(block.timestamp, _from)`, so that only one rate has to be stored. Off-chain consumers reading the event therefore derive a rate and a window that do not match on-chain accrual.

Code Corrected

The `notifyRewardRate()` function and the `NotifyRewardRate` event were removed.



#018 Incoming Bridge Transfer Corrupts Swap's Slippage Check

Security

Low

Version 2

Open

In `Metarouter`, `_v2SwapExactIn()` snapshots the recipient's balance of the output token before the swap and requires the difference measured after the last hop to be at least `minAmountOut`. That difference counts all tokens the recipient received in between, not only what they received in the final swap.

The receiving user could have a pending cross-chain transfer that is deliverable on this chain but has not been credited yet. If that user then selects a swap route containing a token that hands control to a third party on transfer, an attacker can take control mid-swap and deliver the pending transfer into the user's account.

The route can then under-deliver by as much as the bridged amount while still allowing the check to pass, because the balance difference counts both together.

Getting such a token into a pool is not hard, since `PoolFactory.createPool()` is permissionless. The attacker would still need to fund it, or quote well enough, for the off-chain routing to pick it as a preferred route. That routing may well allow it, since users are always protected by `minAmountOut`. However, that assumption does not hold with an incoming transfer.

#019 Missing Gauge Linkage Check in createGauge

Design

Low

Version 1

Open

`GaugeFactory.createGauge()` writes the supplied `_votingRewardsManager` into the new gauge without checking that the manager's immutable `gauge` points back at it. On a mismatch the new gauge's fees can never be collected, since `V2Gauge.collectFees()` only accepts calls from the linked manager and that manager only ever calls `collectFees()` on its own gauge. Only `leafVoter` can create gauges, so this is a misconfiguration risk.

#020 Missing Underflow Checks in _earnedIncentiveOpenInterval

Correctness

Low

Version 1

Open

`VotingRewardsManager._earnedIncentiveOpenInterval()` replays the global checkpoint advance in memory for the `earnedIncentives()` view, flooring the simulated `bias` at zero but not the `slope`. Its on-chain counterpart `_createGlobalRewardPoints()` clamps both. Slope changes that do not exactly cancel the active slope would therefore skew the quote in the wrong direction.

#021 Penalty Timers Can Be Reset Before Activation

Design

Low

Version 1

Open

`V2Gauge.depositFor()` reverts while the effective penalty config is active, so that third parties cannot reset an account's early-unstake timer. `_deposit()` nevertheless sets `depositBlock[_owner] = block.number` while the config is inactive. Hence, an attacker can front-run the transaction that activates the config with dust deposits for every staker. Each of those accounts that claims or withdraws within `minStakeBlocks` of the forced reset loses `penaltyRate` of its accrued emissions, which are forfeited. `penaltyRate` is capped only by `MAX_PIPS`, so the loss can reach 100%. Accounts that wait out the window are unaffected.

#022 Racing Gauge Deployment to Set the Initial Referral Recipient

Design

Low

Version 2

Open 

Neither the creation of pools in `PoolFactory.createPool()` nor of gauges in `DefaultGaugeCreationModule.createGauge()` requires special permissions. The latter requires, among other things, that the attached pool was deployed by a whitelisted factory and that at least one of its tokens is a listed base asset and (for activation) that both tokens are registered in the token registry.

The gauge creator can set an optional referral configuration consisting of a recipient address and a share of the gauge emissions paid to that recipient. The recipient is unrestricted and the share is only bounded by the factory's `maxShareCap`, initialized to 5% but changeable up to 100% by `REFERRAL_ADMIN_ROLE`.

A pool can only ever be linked to one gauge, so the first creator's configuration persists. Once an asset is added as a listed base asset, anyone therefore has an incentive to race to create the gauges of the popular pools built on it to siphon off a share of their LP emissions. `REFERRAL_ADMIN_ROLE` can overwrite the configuration through `GaugeFactory.setReferralConfig()`, but that requires manual intervention per affected gauge.

#023 Router Uses Unbounded Claim Entrypoint

Design

Low

Version 2

Open 

`ClaimsLib.claimGaugeRewards()` claims emissions through the two-argument `claimEmissions()`, which attempts to claim emissions for all the staked NFTs of an account.

Anyone can donate an NFT to an account via function `Gauge.depositFor()`, so an attacker could grow a chosen account's set with dust positions until the claim exceeds the block gas limit. Note that the victim could still claim rewards for each individual `tokenId` with

```
function claimEmissions(
    address _account,
    address _recipient,
    uint256[] calldata _tokenIds)
```

However, this function is not exposed by the router, so a donation could break integrations relying on the router or decrease the UX.

#024 Swaps and Quotes Fail on a Stable Pool with Large Reserves

Design

Low

Version 2

Open 

In `StablePool`, `_k()` evaluates the invariant as `_a * _b`, where both values are in `1e18` of precision:

```
uint256 _a = (_scaledX * _scaledY) / 1e18;
uint256 _b = ((_scaledX * _scaledX) / 1e18 + (_scaledY * _scaledY) / 1e18);
return (_a * _b) / 1e18; // x^3*y + y^3*x
```

For a balanced pool holding `X` whole tokens a side, that product is `2 * X^4 * 1e36`, which overflows if:

```
2 * X^4 * 1e36 > 2**256 - 1  ~=  1.16e77
X^4 > 5.79e40
X > 1.55e10
```

A reserve of 1.55e10 tokens is reachable for a stablecoin with a low unit value. Two Korean won stablecoins can reach it at a relatively unremarkable pool size: 1.55e10 won divided by 1,400 won per USD is about 11 million dollars. Only the first mint is checked against the bound, after which `swap()`, `getK()` and the quote views panic with `Panic(0x11)` and only a burn leaving both sides under the bound succeeds.

#025 Unbounded MEV Tax Can Push the Fee Past the Traded Amount

Design

Low

Version 2

Open

In `PoolFactory`, `getBaseFee()` rejects a fee module value above 10 percent and falls back to `defaultFee`. `getFee()` rejects a MEV tax only above the encodable range of `uint24`, so it accepts a tax of roughly 1677 percent. A total fee above 100 percent reverts `swap()` and both total-fee quotes in `Pool`.

#026 Vote on a Dormant Rewards Contract Corrupts Voting Supply

Correctness

Low

Version 1

Open

`VotingRewardsManager._createGlobalRewardPoints()` fills missing weekly checkpoints in a loop bounded by `MAX_CHECKPOINT_ITERATIONS = 520`. On exhausting that bound `_lastPoint.ts` keeps the 520th weekly boundary rather than `block.timestamp`, yet the code after the loop still adds the veNFT's weight measured at `block.timestamp`:

```
if (_stakeEnd > block.timestamp) {
    _lastPoint.bias += _userRewardPoint.bias;
    _lastPoint.slope += _userRewardPoint.slope;
}
```

That point becomes the newest global checkpoint, and this point is incorrect, since since it includes a voter whose own point is dated later. That stale point becomes `globalCheckpointIndex`, so `_supplyAt()` decays the added weight again while `balanceOfNFTAt()` does not, letting a veNFT's weight exceed `totalSupply()` and inflate the accumulator `claimFees()` pays against. Reaching the cap needs one reward contract untouched for roughly 10 years.

#027 A Third Party Can Make an LP Position Burn Fail

Informational

Version 2

Open

Burning an LP position requires three separate actions, first `DECREASE_CL_LIQUIDITY` to delete the liquidity in the `CLPool` writing the balances to `tokensOwed0` to then `COLLECT_CL_FEES` to receive the tokens owed and finally `BURN_CL_POSITION` to delete the NFT. The router allows batching these actions to simplify their use. `DECREASE_CL_LIQUIDITY` takes the liquidity to remove as an argument, so the multicall must know the exact liquidity amount.

Burning an LP position in `NonfungiblePositionManager.burn()` requires that the position has no liquidity left. Adding liquidity to a position is permissionless, so a griever can add one wei of liquidity to a user's position before the owner's batch operation executes, so that one unit of liquidity stays in place after `DECREASE_CL_LIQUIDITY` is executed making the burn and as a result the whole batch operation revert.

#028 Checkpoint Index Helper Naming Omits Rounding Direction

Informational

Version 1

Open

`VotingCheckpoints._getPastGlobalPointIndex` and `VotingRewardsManager._globalCpIndexAt` have similar functionality and naming, but a subtle difference that their names do not reflect. The former rounds its binary-search midpoint up and returns the greatest checkpoint index at or before the target. The latter rounds down and returns the least index at or after. A caller can pick the wrong helper and land one interval off.

#029 Compiler Version Affected by Known Bug

Informational

Version 1

Open

MetaDEX Application Layer pins the Solidity compiler version to `0.8.30` in `foundry.toml`. That version is affected by the high-severity compiler bug `SOL-2026-1`, a storage-slot collision when `delete` is applied to a transient variable and the code is compiled with the via-IR pipeline. The `[profile.optimized]` and `[profile.test]` profiles set `via_ir = true`, so that precondition is met. However, the bug is not reachable in the current code, since transient variables are only assigned and never deleted.

#030 Cross-Chain Commands Do Not Check Most Hyperlane Inputs

Informational

Version 2

Open

In `CrosschainLib`, `executeCrosschain()` and `bridgeToken()` pass most of their Hyperlane inputs unchecked: the enrolled security module, the commitment, the refund address, the metadata variant, the fee token against `_params.tokenFee`, `_params.messageFee` against the fee token, and the warp route itself, which is neither restricted to a known set nor checked for a remote on the destination domain.

A wrong value in any of these can cause a revert within Hyperlane, or, in the worst case, approve `bridgeToken()` to some malicious contract.

#031 Duplicated Constant and Magic Values in the Gauge Contracts

Informational

Version 1

Open

`Gauge` and `GaugeFactory` each re-declare a local `MAX_PIPS = 1_000_000`, duplicating the constant already defined in `ProtocolConstants`.

Further, the `GaugeFactory` hardcodes `DEFAULT_PENALTY_RATE = 1_000_000` instead of setting it to `MAX_PIPS`. All copies currently hold the same value, but magic dependencies can introduce bugs in future refactoring.

#032 Elapsed Time Is Recomputed in _createGlobalRewardPoints

Informational

Version 1

Code Corrected

In `VotingRewardsManager._createGlobalRewardPoints()` the loop filling the history assigns `_dt = _t_i - _lastTimestamp`, then the bias as `_lastPoint.bias -= _lastPoint.slope * (_t_i - _lastTimestamp).toInt128()` instead of reusing `_dt`.

Code Corrected

The bias update now uses `_dt`.

#033 Exact-Output V2 Routes Do Not Verify the Amount Received

Informational

Version 2

Open

In `SwapsLib`, `v2SwapExactOut()` returns as soon as its swap loop ends, with no check on the amount received. The other three swap handlers all check theirs. `clSwapExactOut()` compares the amount the last pool returned with the amount requested, `clSwapExactIn()` checks the received amount against `minAmountOut`, and `v2SwapExactIn()` requires the recipient's balance to grow by at least `minAmountOut`. The only guard in `v2SwapExactOut()` is `maxAmountIn`, compared against the quoted input amount before any pool is called.

`Pool.swap()` transfers the requested output and only then checks the curve invariant, against its own balances, so the invariant holds whether or not the recipient received the full amount. A final output token that has fees on transfer, is rebasing, or reports success without moving a balance therefore under-delivers silently. Since pools can be permissionlessly created for arbitrary tokens, the scenario is plausible. The same token at an intermediate hop reverts instead, since the next pool computes the input amount from its own balance.

#034 Fee Caching Assumes Block.number Counts the Chain's Blocks

Informational

Version 2

Open

In `DynamicSwapFeeHook`, `_getFee()` keys the cached total fee by `block.number`, and only the first swap under a given key recomputes the dynamic component and consumes the initial fee override. On a chain where `block.number` reports the L1 block instead of the chain's own blocks (e.g. Arbitrum), both therefore happen once per L1 block instead of once per L2 block.

#035 Fee Collection Is Gated on the veNFT's Current Balance

Informational

Version 2

Open

`VotingRewardsManager._claimFees()` collects the gauge's fees on an unbounded claim only while the veNFT still carries weight at `block.timestamp`:

```
if (lastPendingFees0 > 0 || lastPendingFees1 > 0 || (_reachedEnd &&
    _balanceOfNFTAt(_tokenId, block.timestamp) > 0))
```

A veNFT that held weight while the fees accrued but holds none at claim time therefore does not trigger the collection, and is credited its share only once another claim or `flushFees()` collects. Nothing is lost, the claim has to be repeated.

#036 Fee Values Are Indexed in the Dynamic Fee Hook Events

Informational

Version 2

Open

In `IDynamicSwapFeeHook`, every event declares all of its parameters `indexed`, including value fields such as a fee, a fee cap, a scaling factor and the observation window. Marking every parameter as indexed places the value in a log topic instead of the data section, which increases their gas cost. Consumers are likely searching for these events by a pool address, so indexing the value fields does not give any filtering that can be useful in practice.

#037 Gas Optimizations in Pool and PoolOracle

Informational

Version 1

Code Corrected

The MetaDEX Application Layer `Pool` and `PoolOracle` contracts could optimize commonly executed callpaths:

- `Pool._update(uint256,uint256,uint256,uint256)` could emit the local variables `balance0` and `balance1` in event `Sync` instead of reading `reserve0` and `reserve1` from storage, saving two (warm) SLOADs per `swap`, `mint`, `burn`, and `sync`.
- `Pool._update(address,address,uint256)` could skip the `_updateFor` call for the zero address, which never makes use of the `supplyIndex0` and `supplyIndex1` indices.
- `PoolOracle.write` could read `self.cardinality` only after the `_timeElapsed <= _periodSize` early-return check, saving one SLOAD on the common path where no new observation is written.

Code Corrected

All three optimizations were implemented in `Version 2`.

#038 Gas Optimizations in VotingRewardsManager Reward Math

Informational

Version 1

Open

`VotingRewardsManager` runs `_globalCpIndexAt` binary searches over `_globalRewardPointHistory` whose result is already known from the state of the loop or:

- `_computeFeeRewards` could skip the `_lastActiveCp` search when `_globalRewardPointHistory[_gcpEnd].ts <= _stakeEnd`, since it always returns `_gcpEnd`.
- `_computeIncentiveRewards` could skip the `_expiryIndex` search when `_stakeEnd` is past the timestamp at `_params.to`, since it returns `_params.to` and no clip follows.
- `_priceSpan` could skip the `_startIndex` search when `_params.startTs <= _globalRewardPointHistory[_gStart].ts` and the `_endIndex` search when `_params.endTs >= _globalRewardPointHistory[_gEnd].ts`, since they return `_gStart` and `_gEnd`, respectively.

4. `_computeFeeRewards` could reuse `_cursor.currGlobalCp` instead of re-deriving `_gcpStart` by search, since after the first iteration it always equals the previous iteration's `_gcpEnd`.

#039 `getAmountOut` Quotes a Fee the Caller May Not Receive

Informational	Version 2	Open
---------------	-----------	------

In `IPool`, `getAmountOut()` is described only as the amount out for an amount in, but unlike `swap()` and `getAmountOutWithTotalFee()` it does not deduct the MEV tax. This limitation is not described in the natspec.

#040 Hyperlane Dependency Pinned to Outdated Version

Informational	Version 2	Open
---------------	-----------	------

`package.json` pins `@hyperlane-xyz/core` to `10.1.3` while `CrosschainLib` assumes that Hyperlane `11.3.1` or later is getting used with the protocol. For example, `CrosschainLib._hookMetadataFeeToken()` reads a field at offset 86 that was added in `11.0.0`. The library integrates via Hyperlane through a hand-written interface, so nothing catches this at compile time.

#041 Inaccurate Comment on the Emission Cap

Informational	Version 1	Open
---------------	-----------	------

`LeafVoter._accrueSegment()` states that a near-max or sentinel `_cap` is unreachable. However, this comment is incorrect since the `CAP_ADMIN_ROLE` can set the default cap to `type(uint128).max`. A gauge's cap will resolve to the default cap whenever its stored value is the magic constant `DEFAULT_CAP_INDICATOR`, which equals `type(uint128).max` and is what `createGauge()` assigns to every new gauge.

#042 Incomplete Empty-Window Clause

Informational	Version 1	Code Corrected
---------------	-----------	----------------

In `V2Gauge._settleRewards()`, the `_to == _prevEnd` clause that preserves the old rate across an empty window is skipped when the window is future-dated, since `_prevEnd` is then `block.timestamp`. However, in the alternate path the old rate `_oldRate * (_from - _prevEnd) / (_from - _prevEnd)` is still preserved.

Code Corrected

`_settleRewards()` was removed. `V2Gauge` now follows a pull model in which `_advanceRewards()` reads the cumulative reward share from the `LeafVoter`, so the empty-window clause no longer exists.

#043 Low-Level Storage Access in `_createGlobalRewardPoints()`

Informational	Version 1	Open
---------------	-----------	------

The `else` branch of `_createGlobalRewardPoints()` in `VotingRewardsManager` snapshots the fee accumulator with a direct mapping write, `feeRewardPerVotingPowerAt[_globalIndex] =`

`feeRewardPerVotingPower`, instead of the `_snapshotFeeAccumulator()` helper the same function uses when going over weeks to fill the history. Note that the two writes are equivalent and behavior is unchanged.

#044 Misleading Variable Name Xy in `_getAmountOut()`

Informational	Version 1	Code Corrected 
---------------	-----------	--

In `StablePool._getAmountOut()` the return value of `_k()` is stored in a local named `xy`, implying the constant-product invariant $x * y = k$. `StablePool` uses the Solidly stable-swap invariant $x^3 * y + y^3 * x$ instead, so `xy` is not a product of reserves. The value is computed and consumed correctly, so there is no functional impact.

Code Corrected
The local variable was renamed to `_targetK`.

#045 Missing Bounds Check on Program ID for Incentives

Informational	Version 1	Open 
---------------	-----------	--

The `incentives()` and `incentive()` views in `IncentiveStreaming` read caller-supplied program IDs from the `_incentives` mapping without checking they fall in the valid range `1..incentiveCount`. Passing an out-of-range ID returns a zero-initialized struct with no way of signaling that the ID is invalid. This behavior is not documented.

#046 Missing Event Emission in `_claimFees()`

Informational	Version 1	Code Corrected 
---------------	-----------	--

`VotingRewardsManager._claimFees()`, called by the external `claimFees()`, collects gauge fees without emitting a collection event, unlike `flushFees()`, which emits `FeesFlushed` for each collected amount. An off-chain indexer reconstructing collected gauge fees from `FeesFlushed` logs therefore misses fees pulled through the claim path. On-chain accounting is unaffected.

Code Corrected
Fee collection was moved into a shared helper `_settleCollectedFees()` that emits `FeesCollected` for each collected amount.

#047 Native Asset with an ERC-20 Entry Point Makes a Batch Revert

Informational	Version 2	Open 
---------------	-----------	--

In `Metarouter`, `_beginExecution()` snapshots the router's native balance as `address(this).balance - msg.value`, so the snapshot holds whatever native balance the router had before the batch. On a chain where the native asset also has an ERC-20 representation, for example zkSync, Polygon or Celo, a batch that also touches that ERC20 representation will add it to the tracked ERC-20 list.

`_endExecution()` sweeps every tracked ERC-20 in full before it computes the native refund. If the router already held some prior native balances, the tokens will be swept through their ERC-20 entry point, making the computation of the native refund revert:

```
if (_nativeBalance < _nativeBalanceBefore) revert IMetarouter.PreBatchNativeBalanceSpent();
```

#048 No Check That Gauge Address Is as Expected

Informational

Version 2

Open 

`GaugeFactory.createGauge()` computes the gauge address with `predictDeterministicAddress()`, records it and stores it in the linked `VotingRewardsManager`'s immutable `gauge`, then deploys with `cloneDeterministic()` without checking that the deployed address matches.

#049 Oracle Readiness Check Is Stricter than the Revert It Guards

Informational

Version 2

Open 

`DynamicSwapFeeHook._getDynamicFee()` and `DynamicSwapFeeModule._getDynamicFee()` skip the dynamic component when the pool's observation buffer is judged too small to answer the TWAP query.

```
if (_observationCardinality < (_secondsAgo / MIN_SECONDS_AGO)) return 0;
```

The revert being avoided is in the oracle, where a target older than the oldest stored observation is rejected.

```
// ensure that the target is chronologically at or after the oldest observation
require(lte(time, beforeOrAt.blockTimestamp, target), 'OLD');
```

Two things make the guard stricter than that revert requires.

1. `MIN_SECONDS_AGO` is 2, documented as the block time, so the check assumes one observation every two seconds. That is the densest packing the oracle permits. Observations are written at most once per block and only when the tick or liquidity changes, so a pool with sparse trading stores far fewer entries than blocks elapsed and each one spans much more than two seconds. Such a pool can hold a long history in few slots and still be rejected. With the default `secondsAgo` of 600 the guard demands an `observationCardinality` of at least 300, which a pool reaches only after the buffer has been grown and then filled across 300 distinct blocks.
2. The `observe()` call that would produce the revert is already wrapped in a `try`, and the `catch` returns 0. The guard is therefore an optimization rather than a safety requirement, since the revert is handled either way.

The effect is that the dynamic component silently resolves to 0 and the pool charges the base fee, for pools whose oracle would have answered the query. The behavior is indistinguishable from a pool configured to have no dynamic component.

A direct check could read the oldest observation rather than inferring it from cardinality, as `OracleLibrary.getOldestObservationSecondsAgo()` already does by reading `observations[(observationIndex + 1) % observationCardinality]`, falling back to index 0 when that slot is not yet initialized, and comparing the elapsed time against `secondsAgo`. Note the trade-off. The current check is free because `_observationCardinality` already arrives with the `slot0` read, whereas reading the oldest observation adds a cold `SLOAD` to every swap through `beforeSwap()`.

#050 Referral Config Is Read as a Flattened Tuple

Informational

Version 1

Open

`referralConfig` is a public mapping to a `ReferralConfig` struct, so Solidity's auto-generated getter flattens it into a tuple. `V2Gauge.earned()` and `_claimEmissions()` therefore destructure it positionally, which is harder to read and maintain. Adding a field changes the getter's signature and forces every call site to be revisited. The sibling configuration takes the other route, with `penaltyConfig()` and `effectivePenaltyConfig()` as explicit getters that return `PenaltyConfig` memory and let `V2Gauge` read the fields by name.

#051 Single and Bulk Fee Setters Duplicate Validation

Informational

Version 2

Open

In `DynamicSwapFeeHook`, three pairs of setters write the same configuration, once per pool and then in bulk.

- `setCustomFee()` against `_bulkUpdateFees()`
- `setFeeCap()` against `bulkUpdateFeeCaps()`
- `setScalingFactor()` against `bulkUpdateScalingFactors()`

Neither setter in a pair delegates to the other. Each writes the configuration itself and repeats the same sanity checks, which worsens readability and maintainability.

#052 The Fee Resolution Order Is Undocumented

Informational

Version 2

Open

In `DynamicSwapFeeHook`, `_getFirstSwapFee()` resolves the fee of a block's first swap through the decision tree below. Every later swap in that block pays the stored value with any applicable discount applied.

The decision tree is not well documented and contains behavior that contradicts the documentation. The pool's `feeCap` is ignored while `scalingFactor` is zero, so setting a cap alone does not change any fee charged, although `setFeeCap()` accepts it. Function `setScalingFactor()` documents this dependency, but in the other direction. Further, the initial fee circumvents the fee cap entirely:

```
is baseFee the sentinel ZERO_FEE_INDICATOR?
├─ yes: [feeToUse = 0, feeToStore = sentinel]
└─ no: is pool's baseFee 0?
      ├─ yes: base = factory fee (pool's tick spacing)
      └─ no: base = baseFee
            ┌─ is scalingFactor 0?
            │   ├─ yes: cap = defaultFeeCap, scale = defaultScalingFactor
            │   └─ no: cap = pool's feeCap, scale = pool's scalingFactor
            └─ total = min(base + deviation * scale, cap)
                  ┌─ is initialFeeEnabled set?
                  │   └─ no: [feeToUse = discount(total), feeToStore = total]
                  │   └─ yes: what is initialFee?
                  │       └─ 0: [feeToUse = base, feeToStore = total]
                  │       └─ sentinel: [feeToUse = 0, feeToStore = total]
```

```
└─ other: [feeToUse = initialFee, feeToStore = total]
```

#053 The Maximum Fee a Fee Module May Return Is Undocumented

Informational

Version 2

Open 

In `PoolFactory`, `getBaseFee()` accepts a fee module's value only up to a bare `1000`, which is 10 percent in bips:

```
uint256 _customFee = abi.decode(_data, (uint256));
if (_customFee <= 1000) {
    return _customFee;
}
```

Neither `IFeeModule` nor `IPoolFactory` documents that ceiling, while every other fee bound in the system is a named constant. Worse, the MEV tax path of `getFee()` declares no such bound at all, breaking the convention that a fee returned by a module or a hook is sanitized by its caller. `Pool` applies whatever the factory returns without a check of its own, so the bound has to come from the factory.

#054 The per-Interval Swap Count Saturates

Informational

Version 2

Open 

In `CLPoolTape`, the per-interval swap counter stops incrementing once it reaches `type(uint16).max`, so every interval past 65,535 swaps reports the same count into the volatility ring. The width of an interval is defined by the pool's cadence that is set by the owner, so whether the 65,535 swaps in one window are reachable depends entirely on how long that cadence is.

Nothing reads the swap counter today. A future consumer has to account for the saturation and distinguish a count sitting at the cap from a genuinely large one. On an L2 the gas costs of each swap are much cheaper so a cap is reachable in principle, given enough incentive to manipulate.

#055 Unchecked Return Data Length in PoolFactory.getFee()

Informational

Version 1

Code Corrected 

`PoolFactory.getFee()` decodes the fee module's response after checking the success flag:

```
(bool success, bytes memory data) =
    feeModule.excessivelySafeStaticCall(500_000, 32,
        abi.encodeWithSelector(IFeeModule.getFee.selector, _pool));
if (success) {
    uint256 customFee = abi.decode(data, (uint24));
}
```

`excessivelySafeStaticCall` caps the data returned at 32 bytes but copies fewer when the callee returns less, so a fee module returning less than 32 bytes makes `abi.decode` revert. This violates the invariant that a failure of the fee module should not revert a swap. Similar call sites in the codebase check the length before decoding.

Code Corrected

The return data length is now checked before decoding in `Version 2`.

#056 Undocumented Viewer Deviations

Informational	Version 2	Open
---------------	-----------	------

`earnedFees()` and `earnedIncentives()` return figures that deviate from the amounts their respective claim entryptoints would transfer in the same block. Neither view is read on chain, however integrators might be affected:

Fees.

- The pending fees after the last user checkpoint are calculated as `weight * pending / supply`, whereas a claim routes the same amount through `_settleCollectedFees()` and `_notifyFeesAmount()` buffering amounts below one accumulator unit, gating per token, and truncating twice. One accumulator unit is `totalSupply / PRECISION`. So with a total supply of 100M voting power at least 1e8 raw units of the fee token must accrue before anything is credited. With a six-decimal token such as USDC that is 100.000000 USDC. Hence for a sole voter holding that voting power, with 99.999999 pending fees the whole amount is buffered, while the view reports 99.999999. With 199.999999 pending the viewer returns 199.999999, while `claimFees()` pays out 100.000000.
- The value `lastFeeUpdate` is ignored. The claim endpoint buffers any new accrual when the accumulator has already been advanced in the same block, and pays out nothing.

Incentives.

- `earnedIncentives()` prices the open window at reference timestamp `min(block.timestamp, program.end) - 1`, whereas `_claimIncentives()` closes the window into intervals capped at a week and prices each from its own right edge.
- The view validates no program id, returning zero where the claim reverts with `InvalidProgramId`.

#057 Unreachable K == Xy Branch in `_get_y()`

Informational	Version 1	Code Corrected
---------------	-----------	----------------

In `StablePool._get_y()`, the `if (k == xy)` check sits inside the `if (k < xy)` branch, so it is always false and the following `return y` is dead code. The equal case is handled in the `else` branch, so there is no functional impact.

Code Corrected

The unreachable equality check was removed.

#058 Unused Data Argument in `Checkpoint()`

Informational	Version 1	Open
---------------	-----------	------

`VotingRewardsManager.checkpoint()` declares a `bytes calldata _data` parameter and forwards it to `_checkpoint()` in `VotingCheckpoints`, which never reads it. The value passed by `LeafVoter._checkpointReward()` is discarded on every checkpoint.

#059 Unused Return Value of `_notifyFeesAmount()`

Informational

Version 1

Code Corrected 

`FeeDistribution` has two instances of `_notifyFeesAmount()`, both returning the credited token0 and token1 amounts. The no-arg overload `_notifyFeesAmount()` computes these amounts from `gauge.pendingFees()`, but all of its callers discard the return values. Only the two-arg overload `_notifyFeesAmount(uint256,uint256)` returns the amounts, and return values are later consumed in `flushFees()` and `_claimFees()`. The no-arg overload's return values are therefore dead.

Code Corrected

The no-argument overload of `_notifyFeesAmount()` was removed in `Version 2`.

8 Open Questions

Here, we list open questions that came up during the assessment and that we would like to clarify to ensure that no important information is missing.

OQ1 Is a Zero Volatility Range Intended for Single-Swap Intervals?

Open Question	Version 2
---------------	-----------

In `CLPoolTape`, `record()` seeds both `intervalMaxTick` and `intervalMinTick` from the incoming swap tick whenever `intervalSwapCount` is zero, so min and max tick only cover the swaps inside:

```
if (_poolAccumulator.intervalSwapCount == 0) {
    _poolAccumulator.intervalMaxTick = _data.tick;
    _poolAccumulator.intervalMinTick = _data.tick;
} else {
    // ... widen the extremes from there
}
```

`_pushVolatility()` stores `intervalMaxTick - intervalMinTick` as the interval's range, so an interval holding a single swap commits a range of zero however far the price moved since the previous interval. A pool whose swaps are spaced further apart than the cadence commits an observation on every swap, so every one of its intervals holds a single swap and the whole ring fills with zero ranges. The movement across the boundary is not lost, since the same push stores the distance between the closing tick and `intervalOpenTick`.

Is it intended that the range covers only the movement between swaps inside an interval, leaving the movement across the boundary to the distance term?

OQ2 Why Is the Fee Inversion in the Pool and Not the Fee Module?

Open Question	Version 2
---------------	-----------

In `Pool`, `getAmountInWithTotalFee()` inverts the gross amount by iteration, because it cannot know the fee equation. The fee module does, and even where no closed-form inverse exists it could implement its own algorithm. Have you considered implementing `getAmountInWithTotalFee()` in the fee module and swap hook interfaces instead?

9 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.